```
// 软件名称：墨译漫画小说智能翻译软件
// 版本号：V1.1.0
// 著作权人：薛朝坤
// 本文件为计算机软件著作权登记源程序鉴别材料（前30页+后30页）。
//
// ==================== 文件：entry/src/main/ets/service/PageTranslateService.ets ====================
import fs from '@ohos.file.fs';
import { image } from '@kit.ImageKit';
import { preferences } from '@kit.ArkData';
import { OcrRegion, PageTranslation, TranslateStatus, makePageTranslation } from 'coremodel';
import { EngineType, TranslateRequest, Translator } from 'coretranslate';
import { OcrLang, OcrService } from 'coreocr';
import { Log, toMediaUri } from 'commons';
import { util } from '@kit.ArkTS';
import { AppEnv } from '../common/AppEnv';
import { PageRepository } from '../data/PageRepository';
import { SettingsKeys, SettingsRepository } from '../data/SettingsRepository';
import { OcrModelService } from './OcrModelService';
import { GlossaryService } from './GlossaryService';
import { OcrRecService } from './OcrRecService';

const TAG = 'PageTranslateService';
const PREF_FILE = 'starflow_translate';
const KEY_OCR_LANG = 'ocr_lang';
const KEY_ENGINE = 'text_engine';

/**
 * 单页翻译闭环服务（T025 / plan D6/D13）：
 * 解码页图 → Core Vision OCR 行级识别 → 引擎逐行串行翻译 → 按序回填 →
 * 落库 page_translation（只存译文文本 + 原始坐标，FR-014）。
 *
 * 说明：PixelMap 非 Sendable，无法从 taskPool 返回，故图像解码走主线程
 * 异步链（image.createImageSource 为异步 API，不阻塞帧）；OCR 与引擎调用
 * 亦运行在 ArkTS 线程。
 */
export enum TranslatePhase {
  OCR = 0,
  TRANSLATE = 1,
  SAVING = 2
}

/** 单页翻译的可观察进度（OCR → 逐行翻译 → 落库）。 */
export class TranslateProgress {
  phase: TranslatePhase = TranslatePhase.OCR;
  done: number = 0;
  total: number = 0;
  percent: number = 0;
  detail: string = '';
}
```

```
export type TranslateProgressCallback = (p: TranslateProgress) => void;

export class PageTranslateService {
  private static dictText: string = '';
  /** 页内行级译文缓存：漫画气泡重复率高，命中可直接跳过网络/模型。 */
  private static lineCache: Map<string, string> = new Map<string, string>();
  private static readonly LINE_CACHE_MAX = 600;

  private static async configureOcr(): Promise<void> {
    const tier = await SettingsRepository.getString(SettingsKeys.OCR_ENGINE_TIER, 'system');
    const service = OcrService.getInstance();
    if (tier !== 'local') {
      service.setLocalEnabled(false);
      return;
    }
    if (!OcrModelService.localModelsReady()) {
      throw new Error('本地 OCR 模型未下载：请到「设置 → OCR」下载检测与识别模型');
    }
    if (PageTranslateService.dictText.length === 0) {
      const bytes = await
AppEnv.getInstance().getContext().resourceManager.getRawFileContent('ppocrv6_dict.txt');
      const decoder: util.TextDecoder = util.TextDecoder.create('utf-8');
      PageTranslateService.dictText = decoder.decodeToString(bytes);
    }
    const models = OcrModelService.list();
    service.configureLocal(OcrModelService.path(models[0]), OcrModelService.path(models[1]),
      PageTranslateService.dictText);
    service.setLocalEnabled(true);
    OcrRecService.initNative(OcrModelService.path(models[0]), OcrModelService.path(models[1]));
  }

  /** 启动预热：系统 OCR 初始化或本地 OCR 会话加载，失败只记日志。 */
  static async prewarm(): Promise<void> {
    await AppEnv.ready();
    await PageTranslateService.configureOcr();
    await OcrService.getInstance().prewarmLocal();
    Log.info(TAG, 'OCR prewarm done, local=' + OcrService.getInstance().isLocalEnabled());
  }

  static async translatePage(pageId: number, engineType: EngineType,
    onProgress?: TranslateProgressCallback): Promise<PageTranslation> {
    return PageTranslateService.run(pageId, engineType, onProgress);
  }

  static async retryPage(pageId: number, engineType: EngineType,
    onProgress?: TranslateProgressCallback): Promise<PageTranslation> {
    return PageTranslateService.run(pageId, engineType, onProgress);
  }

  private static async run(pageId: number, engineType: EngineType,
```

```
    onProgress?: TranslateProgressCallback): Promise<PageTranslation> {
    await AppEnv.ready();
    const page = await PageRepository.getPage(pageId);
    if (page === undefined) {
      throw new Error('页面记录不存在');
    }
    const lang = await PageTranslateService.loadOcrLang();
    const targetLang = await PageTranslateService.loadTargetLang();
    await GlossaryService.load();
    const tStart = Date.now();
    let pixelMap: image.PixelMap | undefined = undefined;
    try {
      PageTranslateService.emitProgress(onProgress, TranslatePhase.OCR, 2, 0, 0, '准备 OCR');
      await PageTranslateService.configureOcr();

      // 本地 OCR 直接读文件；系统 OCR 才需要先解码 PixelMap。
      const localOcr = OcrService.getInstance().isLocalEnabled();
      if (!localOcr) {
        PageTranslateService.emitProgress(onProgress, TranslatePhase.OCR, 8, 0, 0, '解码页面');
        pixelMap = await PageTranslateService.decode(page.imagePath);
      }
      PageTranslateService.emitProgress(onProgress, TranslatePhase.OCR, 15, 0, 0, '识别文字');
      const tOcr = Date.now();
      // ONNX Runtime 的环境/会话在 TaskPool 中会与其他 worker 冲突
      // （Session is closed / OnnxEnvironment already initialized），
      // 当前版本回到主线程；后续用单一专用 Worker 再迁移。
      let regions: Array<OcrRegion> = [];
      if (localOcr) {
        try {
          regions = await OcrRecService.recognizePage(page.imagePath,
            PageTranslateService.dictText);
        } catch (e) {
          Log.warn(TAG, 'native det/rec failed, fallback JS: ' + e);
        }
        if (regions.length === 0) {
          regions = await OcrService.getInstance().recognize(undefined, lang, page.imagePath);
        }
      } else {
        regions = await OcrService.getInstance().recognize(pixelMap, lang, page.imagePath);
      }
      const ocrMs = Date.now() - tOcr;
      PageTranslateService.emitProgress(onProgress, TranslatePhase.TRANSLATE, 35, 0,
regions.length,
        '识别完成 ' + regions.length + ' 行');

      const translator: Translator = AppEnv.getInstance().engineRegistry.get(engineType);
      const translations: Array<string> = [];
      for (let i = 0; i < regions.length; i++) {
        translations.push('');
      }
```

```
      let failed = 0;
      let firstError = '';
      let completed = 0;
      const tTrans = Date.now();
      const translateLine = async (idx: number): Promise<void> => {
        const region = regions[idx];
        const source = GlossaryService.preReplace(region.text);
        const remembered = GlossaryService.lookup(source);
        if (remembered !== undefined) {
          translations[idx] = remembered;
          return;
        }
        const req: TranslateRequest = { text: source, from: 'auto', to: targetLang };
        const key = engineType + '|' + targetLang + '|' + source;
        const cached = PageTranslateService.lineCache.get(key);
        if (cached !== undefined) {
          translations[idx] = cached;
          return;
        }
        const translated = await translator.translate(req);
        translations[idx] = translated;
        if (PageTranslateService.lineCache.size >= PageTranslateService.LINE_CACHE_MAX) {
          PageTranslateService.lineCache.clear();
        }
        PageTranslateService.lineCache.set(key, translated);
      };
      const markProgress = (n: number): void => {
        completed = Math.min(regions.length, completed + n);
        const pct = regions.length === 0 ? 95
          : 35 + Math.round(60 * completed / regions.length);
        PageTranslateService.emitProgress(onProgress, TranslatePhase.TRANSLATE, pct,
          completed, regions.length, '翻译 ' + completed + '/' + regions.length + ' 行');
      };

      if (engineType === EngineType.LOCAL_LLAMA && regions.length > 1) {
        // 本地 llama 串行成本高：按空间位置合并成气泡/段落，一次生成后按换行拆回。
        const groups = PageTranslateService.buildRegionGroups(regions);
        for (const group of groups) {
          let assigned = false;
          if (group.length > 1) {
            const text = group.map((idx: number): string =>
GlossaryService.preReplace(regions[idx].text)).join('\n');
            const key = 'batch|' + engineType + '|' + targetLang + '|' + text;
            try {
              let joined = PageTranslateService.lineCache.get(key);
              if (joined === undefined) {
                joined = await translator.translate({ text: text, from: 'auto', to: targetLang
});
                if (PageTranslateService.lineCache.size >= PageTranslateService.LINE_CACHE_MAX)
{
```

```
                PageTranslateService.lineCache.clear();
              }
              PageTranslateService.lineCache.set(key, joined);
            }
            const lines = joined.split('\n').map((s: string): string => s.trim())
              .filter((s: string): boolean => s.length > 0);
            if (lines.length === group.length) {
              for (let k = 0; k < group.length; k++) {
                translations[group[k]] = lines[k];
              }
              assigned = true;
            }
          } catch (e) {
            Log.warn(TAG, 'batch translate failed, fallback per-line: ' + e);
          }
        }
        if (!assigned) {
          for (const idx of group) {
            try {
              await translateLine(idx);
            } catch (e) {
              failed++;
              if (firstError.length === 0) {
                firstError = PageTranslateService.readableError(e);
              }
            }
          }
        }
        markProgress(group.length);
      }
    } else {
      // 网络引擎 6 路并发；单行引擎或只有一行时走逐行。
      const concurrency = Math.max(1, Math.min(engineType === EngineType.LOCAL_LLAMA ? 1 : 6,
        regions.length));
      let cursor = 0;
      const worker = async (): Promise<void> => {
        while (cursor < regions.length) {
          const idx = cursor++;
          try {
            await translateLine(idx);
          } catch (e) {
            failed++;
            if (firstError.length === 0) {
              firstError = PageTranslateService.readableError(e);
            }
          }
          markProgress(1);
        }
      };
      const workers: Array<Promise<void>> = [];
```

```
      for (let i = 0; i < concurrency; i++) {
        workers.push(worker());
      }
      await Promise.all(workers);
    }
    Log.info(TAG, 'page ' + pageId + ' timings: ocr=' + ocrMs + 'ms, translate='
      + (Date.now() - tTrans) + 'ms, total=' + (Date.now() - tStart) + 'ms');

    const pt = makePageTranslation(pageId);
    pt.engineType = engineType as number;
    pt.ocr = regions;
    pt.translations = translations;
    if (regions.length === 0) {
      pt.status = TranslateStatus.PARTIAL_FAILED;
      pt.failReason = '未识别到任何文字行';
    } else if (failed > 0) {
      pt.status = TranslateStatus.PARTIAL_FAILED;
      pt.failReason = `${failed}/${regions.length} 行翻译失败: ${firstError}`;
    } else {
      pt.status = TranslateStatus.DONE;
    }
    PageTranslateService.emitProgress(onProgress, TranslatePhase.SAVING, 98,
      completed, regions.length, '保存译文');
    await PageRepository.saveTranslation(pt);
    const sources: Array<string> = [];
    for (const r of regions) {
      sources.push(r.text);
    }
    GlossaryService.rememberPage(sources, translations).catch((e: Object) => {
      Log.warn(TAG, 'remember translation failed: ' + e);
    });
    PageTranslateService.emitProgress(onProgress, TranslatePhase.SAVING, 100,
      completed, regions.length, '完成');
    Log.info(TAG, `page ${pageId}: ${regions.length} lines, ${failed} failed`);
    return pt;
  } catch (e) {
    const reason = PageTranslateService.readableError(e);
    const pt = makePageTranslation(pageId);
    pt.engineType = engineType as number;
    pt.status = TranslateStatus.PARTIAL_FAILED;
    pt.failReason = reason;
    try {
      await PageRepository.saveTranslation(pt);
    } catch (ignore) {
      // 落库失败不掩盖原始错误
    }
    Log.error(TAG, `page ${pageId} failed: ${reason}`);
    throw new Error(reason);
  } finally {
    if (pixelMap !== undefined) {
```

```
      try {
        pixelMap.release();
      } catch (ignore) {
        // release 失败忽略
      }
    }
  }
}

/**
 * 解码页图：沙箱绝对路径需先转媒体 URI（Image/ImageSource 不认裸路径），
 * 失败再退回 fd 打开（目录授权的 content:// 等场景）。
 */
static async decode(path: string): Promise<image.PixelMap> {
  let source: image.ImageSource | undefined = undefined;
  let file: fs.File | undefined = undefined;
  try {
    try {
      source = image.createImageSource(toMediaUri(path));
    } catch (e) {
      source = undefined;
    }
    if (source === undefined || source === null) {
      file = fs.openSync(path, fs.OpenMode.READ_ONLY);
      source = image.createImageSource(file.fd);
    }
    if (source === undefined || source === null) {
      throw new Error('无法解码图片');
    }
    const pm = await source.createPixelMap();
    if (pm === undefined || pm === null) {
      throw new Error('无法创建像素图');
    }
    return pm;
  } finally {
    if (source !== undefined) {
      try {
        source.release();
      } catch (ignore) {
      }
    }
    if (file !== undefined) {
      try {
        fs.closeSync(file);
      } catch (ignore) {
      }
    }
  }
}
```

```
  /** 漫画页目标语言：沿用设置页 target_lang，不再硬编码中文。 */
  static async loadTargetLang(): Promise<string> {
    try {
      const raw = await SettingsRepository.getString(SettingsKeys.TARGET_LANG, 'zh');
      return raw.length > 0 ? raw : 'zh';
    } catch (e) {
      return 'zh';
    }
  }

  static async loadOcrLang(): Promise<OcrLang> {
    try {
      const ctx = AppEnv.getInstance().getContext();
      const pref = await preferences.getPreferences(ctx, PREF_FILE);
      const raw = pref.getSync(KEY_OCR_LANG, OcrLang.JPN as string) as string;
      if (raw === OcrLang.ENG) {
        return OcrLang.ENG;
      }
      if (raw === OcrLang.KOR) {
        return OcrLang.KOR;
      }
      if (raw === OcrLang.CHN) {
        return OcrLang.CHN;
      }
      return OcrLang.JPN;
    } catch (e) {
      return OcrLang.JPN;
    }
  }

  static async saveOcrLang(lang: OcrLang): Promise<void> {
    try {
      const ctx = AppEnv.getInstance().getContext();
      const pref = await preferences.getPreferences(ctx, PREF_FILE);
      pref.putSync(KEY_OCR_LANG, lang as string);
      await pref.flush();
    } catch (e) {
      // 偏好保存失败不影响当次翻译
    }
  }

  /** Persisted default translation engine (Bing when unset). */
  static async loadEngineType(): Promise<EngineType> {
    try {
      const ctx = AppEnv.getInstance().getContext();
      const pref = await preferences.getPreferences(ctx, PREF_FILE);
      const raw = pref.getSync(KEY_ENGINE, EngineType.BING as number) as number;
      if (raw === EngineType.OPENAI_COMPAT) {
        return EngineType.OPENAI_COMPAT;
      }
```

```
    if (raw === EngineType.LOCAL_LLAMA) {
      return EngineType.LOCAL_LLAMA;
    }
    return EngineType.BING;
  } catch (e) {
    return EngineType.BING;
  }
}

static async saveEngineType(type: EngineType): Promise<void> {
  try {
    const ctx = AppEnv.getInstance().getContext();
    const pref = await preferences.getPreferences(ctx, PREF_FILE);
    pref.putSync(KEY_ENGINE, type as number);
    await pref.flush();
  } catch (e) {
    // ignore
  }
}

/** 规则气泡/段落合并：垂直间距小且水平重叠的相邻行视为同组。 */
private static buildRegionGroups(regions: Array<OcrRegion>): Array<Array<number>> {
  const groups: Array<Array<number>> = [];
  let current: Array<number> = [];
  let chars = 0;
  for (let i = 0; i < regions.length; i++) {
    const box = regions[i].box;
    let same = false;
    if (current.length > 0) {
      const prev = regions[current[current.length - 1]].box;
      const vgap = box.y - (prev.y + prev.h);
      const overlap = Math.min(prev.x + prev.w, box.x + box.w) - Math.max(prev.x, box.x);
      const minW = Math.min(prev.w, box.w);
      same = vgap <= Math.max(8, prev.h * 0.8) && overlap > minW * 0.25
        && current.length < 4 && chars + regions[i].text.length <= 600;
    }
    if (!same && current.length > 0) {
      groups.push(current);
      current = [];
      chars = 0;
    }
    current.push(i);
    chars += regions[i].text.length + 1;
  }
  if (current.length > 0) {
    groups.push(current);
  }
  return groups;
}
```

```
  private static emitProgress(cb: TranslateProgressCallback | undefined, phase: TranslatePhase,
    percent: number, done: number, total: number, detail: string): void {
    if (cb === undefined) {
      return;
    }
    const p = new TranslateProgress();
    p.phase = phase;
    p.percent = Math.max(0, Math.min(100, percent));
    p.done = done;
    p.total = total;
    p.detail = detail;
    try {
      cb(p);
    } catch (e) {
      Log.warn(TAG, 'progress callback failed: ' + e);
    }
  }

  static readableError(e: Object): string {
    const err = e as Error;
    if (err !== undefined && err !== null && err.message !== undefined && err.message.length >
0) {
      return err.message;
    }
    return '页面翻译失败';
  }
}
// ==================== 文件: coreocr/src/main/ets/LocalOcrEngine.ets ====================
import { image } from '@kit.ImageKit';
import fs from '@ohos.file.fs';
import { OnnxEnvironment, OnnxTensor, OnnxResult, OnnxSession, OnnxValue, TensorInfo } from
'@ohos/onnxruntime';
import { Log, toMediaUri } from 'commons';
import { OcrRegion, makeOcrRegion, makePixelRect } from 'coremodel';
const TAG = 'LocalOcrEngine';
// 768 足够识别漫画气泡文字，同时比 960 少约 36% 检测像素；
// 实测把整页 OCR 从 2.6s 降到约 1.7s。
const DET_MAX_SIDE = 768;
const DET_THRESH = 0.3;
const DET_MIN_SIZE = 4;
const REC_HEIGHT = 48;
export interface LocalOcrConfig {
    detPath: string;
    recPath: string;
    dictText: string;
}
class PagePixels {
    width: number = 0;
    height: number = 0;
    rgba: Uint8Array = new Uint8Array(0);
```

```
}
class CandidateBox {
    x: number = 0;
    y: number = 0;
    w: number = 0;
    h: number = 0;
}
/**
 * 本地 PP-OCRv6 推理（ONNX Runtime OHOS NAPI）：det 文本检测 + rec CTC 识别。
 * 模型由用户在设置中可选下载；字典随安装包（上游 ppocrv6_dict.txt）。
 * 后处理做了轻量实现：3x3 膨胀 + 连通域 + 轴对齐框，适合漫画气泡文本行。
 */
export class LocalOcrEngine {
    private static instance: LocalOcrEngine = new LocalOcrEngine();
    private detSession: OnnxSession | undefined = undefined;
    private recSession: OnnxSession | undefined = undefined;
    private dict: Array<string> = [];
    private envReady: boolean = false;
    private configKey: string = '';
    static getInstance(): LocalOcrEngine {
        return LocalOcrEngine.instance;
    }
    isReady(): boolean {
        return this.detSession !== undefined && this.recSession !== undefined;
    }
    /** 加载/复用会话；配置变化才重建。 */
    async init(config: LocalOcrConfig): Promise<void> {
        const key = config.detPath + '|' + config.recPath;
        if (this.isReady() && key === this.configKey) {
            return;
        }
        this.release();
        if (!this.envReady) {
            try {
                OnnxEnvironment.InitEnvironment();
            }
            catch (e) {
                const msg = (e as Error).message ?? '';
                // ONNX Runtime 环境是进程级的；另一个线程初始化过后这里会抛
                // "already initialized"，此时直接复用即可，不能当成失败。
                if (msg.indexOf('already initialized') < 0) {
                    throw new Error(msg.length > 0 ? msg : 'ONNX Runtime 初始化失败');
                }
                Log.warn(TAG, 'ORT environment already initialized by another thread');
            }
            // 张量构造自检：验证 NAPI 环境/分配器已就绪（否则会在构造时空指针）。
            const probe = new OnnxTensor(new ArrayBuffer(12), [3], 0);
            probe.Close();
            this.envReady = true;
            Log.info(TAG, 'ORT environment ready, version=' + OnnxEnvironment.GetVersion());
```

```
        }
        const t0 = Date.now();
        try {
            this.detSession = OnnxEnvironment.CreateSessionFromModelFile(config.detPath);
            this.recSession = OnnxEnvironment.CreateSessionFromModelFile(config.recPath);
        }
        catch (e) {
            this.release();
            const err = e as Error;
            const detail = err !== undefined && err.message !== undefined ? err.message :
JSON.stringify(e);
            Log.error(TAG, 'OCR model load failed: ' + detail);
            throw new Error('本地 OCR 模型加载失败: ' + detail + '（请重新下载 det/rec 模型）');
        }
        const lines = config.dictText.split('\n');
        // CTC class 0 = blank; 字典文件第一行才是第一个真实字符，需前置 blank。
        const dict: Array<string> = [''];
        for (const line of lines) {
            const ch = line.endsWith('\r') ? line.substring(0, line.length - 1) : line;
            if (ch.length > 0) {
                dict.push(ch);
            }
        }
        this.dict = dict;
        this.configKey = key;
        Log.info(TAG, 'OCR sessions ready in ' + (Date.now() - t0) + 'ms, dict=' + dict.length);
    }
    release(): void {
        try {
            if (this.detSession !== undefined) {
                this.detSession.Close();
            }
        }
        catch (e) {
        }
        try {
            if (this.recSession !== undefined) {
                this.recSession.Close();
            }
        }
        catch (e) {
        }
        this.detSession = undefined;
        this.recSession = undefined;
        this.dict = [];
        this.configKey = '';
    }
    /** 仅跑 det，返回扁平框 [x,y,w,h,...]；供 Entry 原生 rec 异步使用。 */
    async detectBoxes(pagePath: string): Promise<Array<number>> {
        if (this.detSession === undefined) {
```

```
            throw new Error('本地 OCR 检测模型尚未初始化');
        }
        const page = await this.loadPixels(pagePath);
        const boxes = this.detect(page);
        const out: Array<number> = [];
        for (const box of boxes) {
            out.push(box.x);
            out.push(box.y);
            out.push(box.w);
            out.push(box.h);
        }
        return out;
    }

    async recognize(pagePath: string): Promise<Array<OcrRegion>> {
        if (!this.isReady()) {
            throw new Error('本地 OCR 尚未初始化');
        }
        const page = await this.loadPixels(pagePath);
        if (page.width <= 0 || page.height <= 0) {
            throw new Error('页面图像解码失败');
        }
        const t0 = Date.now();
        const boxes = this.detect(page);
        Log.info(TAG, 'det boxes=' + boxes.length + ' in ' + (Date.now() - t0) + 'ms');
        const regions: Array<OcrRegion> = [];
        for (const box of boxes) {
            try {
                const text = this.recognizeBox(page, box);
                const trimmed = text.trim();
                const area = box.w * box.h;
                const pageArea = page.width * page.height;
                // 过滤整页大框，以及“单字却占大面积”的伪检测（暗色背景/线稿误检）。
                if (trimmed.length === 0 || area > pageArea * 0.15
                    || (trimmed.length <= 2 && area > pageArea * 0.04)) {
                    Log.warn(TAG, 'skip suspicious box ' + box.w + 'x' + box.h + ' text=' +
trimmed);
                    continue;
                }
                regions.push(makeOcrRegion(trimmed, makePixelRect(box.x, box.y, box.w, box.h)));
            }
            catch (e) {
                Log.warn(TAG, 'rec box failed: ' + JSON.stringify(e));
            }
        }
        regions.sort((a: OcrRegion, b: OcrRegion): number => {
            if (Math.abs(a.box.y - b.box.y) > 12) {
                return a.box.y - b.box.y;
            }
            return a.box.x - b.box.x;
```

```
        });
        Log.info(TAG, 'local OCR lines=' + regions.length + ', total=' + (Date.now() - t0) +
'ms');
        return regions;
    }
    private async loadPixels(pagePath: string): Promise<PagePixels> {
        const out = new PagePixels();
        let source: image.ImageSource | undefined = undefined;
        let file: fs.File | undefined = undefined;
        try {
            try {
                source = image.createImageSource(toMediaUri(pagePath));
            }
            catch (e) {
                source = undefined;
            }
            if (source === undefined || source === null) {
                file = fs.openSync(pagePath, fs.OpenMode.READ_ONLY);
                source = image.createImageSource(file.fd);
            }
            if (source === undefined || source === null) {
                throw new Error('无法打开页面图片');
            }
            const pm = await source.createPixelMap();
            if (pm === undefined || pm === null) {
                throw new Error('无法创建页面像素图');
            }
            const info = await pm.getImageInfo();
            out.width = info.size.width;
            out.height = info.size.height;
            const buffer = new ArrayBuffer(out.width * out.height * 4);
            await pm.readPixelsToBuffer(buffer);
            out.rgba = new Uint8Array(buffer);
            pm.release();
            return out;
        }
        finally {
            if (source !== undefined) {
                source.release();
            }
            if (file !== undefined) {
                fs.closeSync(file);
            }
        }
    }
    private detect(page: PagePixels): Array<CandidateBox> {
        let scale = 1;
        const maxSide = Math.max(page.width, page.height);
        if (maxSide > DET_MAX_SIDE) {
            scale = DET_MAX_SIDE / maxSide;
```

```
        }
        const detW = Math.max(32, Math.round(page.width * scale / 32) * 32);
        const detH = Math.max(32, Math.round(page.height * scale / 32) * 32);
        const input = new Float32Array(3 * detH * detW);
        this.resizeToCHW(page, detW, detH, input);
        const detBuffer = new ArrayBuffer(input.length * 4);
        new Float32Array(detBuffer).set(input);
        const detShape: number[] = [1, 3, detH, detW];
        const tensor = new OnnxTensor(detBuffer, detShape, 0);
        let result: OnnxResult | undefined = undefined;
        try {
            const inputs: Record<string, OnnxValue> = {};
            inputs['x'] = tensor;
            result = this.detSession!.Run(inputs);
            const outName = this.detSession!.GetOutputNames()[0];
            const outValue = result.GetValueByName(outName);
            if (outValue.GetType() !== 1) {
                return [];
            }
            const outTensor = outValue as OnnxTensor;
            const info: TensorInfo = outTensor.GetInfo();
            const shape: number[] = info.GetShape();
            const predH = shape[shape.length - 2];
            const predW = shape[shape.length - 1];
            const data = new Float32Array(outTensor.GetDataAsArrayBuffer());
            return this.components(data, predW, predH, page.width, page.height);
        }
        finally {
            tensor.Close();
            if (result !== undefined) {
                result.Close();
            }
        }
    }
    /** 3x3 膨胀 + 连通域 + 轴对齐框（映射回原图坐标）。 */
    private components(data: Float32Array, predW: number, predH: number, srcW: number, srcH:
number): Array<CandidateBox> {
        const mask = new Uint8Array(predW * predH);
        for (let i = 0; i < mask.length; i++) {
            mask[i] = data[i] > DET_THRESH ? 1 : 0;
        }
        // 可分离膨胀（半径 3），把相邻笔画/字符连成文本行；竖排日文也能连上。
        const radius = 3;
        const tmp = new Uint8Array(mask.length);
        for (let y = 0; y < predH; y++) {
            for (let x = 0; x < predW; x++) {
                if (mask[y * predW + x] === 0) {
                    continue;
                }
                const x0 = Math.max(0, x - radius);
```

```
            const x1 = Math.min(predW - 1, x + radius);
            for (let nx = x0; nx <= x1; nx++) {
                tmp[y * predW + nx] = 1;
            }
        }
    }
    const dilated = new Uint8Array(mask.length);
    for (let y = 0; y < predH; y++) {
        for (let x = 0; x < predW; x++) {
            if (tmp[y * predW + x] === 0) {
                continue;
            }
            const y0 = Math.max(0, y - radius);
            const y1 = Math.min(predH - 1, y + radius);
            for (let ny = y0; ny <= y1; ny++) {
                dilated[ny * predW + x] = 1;
            }
        }
    }
    const visited = new Uint8Array(dilated.length);
    const stack: Array<number> = [];
    const boxes: Array<CandidateBox> = [];
    const sx = srcW / predW;
    const sy = srcH / predH;
    for (let i = 0; i < dilated.length; i++) {
        if (dilated[i] === 0 || visited[i] === 1) {
            continue;
        }
        let minX = predW;
        let maxX = 0;
        let minY = predH;
        let maxY = 0;
        let count = 0;
        stack.push(i);
        visited[i] = 1;
        while (stack.length > 0) {
            const idx = stack.pop() as number;
            const x = idx % predW;
            const y = Math.floor(idx / predW);
            count++;
            if (x < minX) {
                minX = x;
            }
            if (x > maxX) {
                maxX = x;
            }
            if (y < minY) {
                minY = y;
            }
            if (y > maxY) {
```

```
                maxY = y;
            }
            if (x > 0 && dilated[idx - 1] === 1 && visited[idx - 1] === 0) {
                visited[idx - 1] = 1;
                stack.push(idx - 1);
            }
            if (x + 1 < predW && dilated[idx + 1] === 1 && visited[idx + 1] === 0) {
                visited[idx + 1] = 1;
                stack.push(idx + 1);
            }
            if (y > 0 && dilated[idx - predW] === 1 && visited[idx - predW] === 0) {
                visited[idx - predW] = 1;
                stack.push(idx - predW);
            }
            if (y + 1 < predH && dilated[idx + predW] === 1 && visited[idx + predW] === 0) {
                visited[idx + predW] = 1;
                stack.push(idx + predW);
            }
        }
        if (count < 12) {
            continue;
        }
        const box = new CandidateBox();
        box.x = Math.max(0, Math.floor(minX * sx));
        box.y = Math.max(0, Math.floor(minY * sy));
        box.w = Math.min(srcW - box.x, Math.ceil((maxX - minX + 1) * sx));
        box.h = Math.min(srcH - box.y, Math.ceil((maxY - minY + 1) * sy));
        if (box.w < DET_MIN_SIZE || box.h < DET_MIN_SIZE) {
            continue;
        }
        // unclip 近似：向外扩 12%，把紧贴的笔画/气泡边距包进识别框。
        const ex = Math.round(box.w * 0.12);
        const ey = Math.round(box.h * 0.12);
        box.x = Math.max(0, box.x - ex);
        box.y = Math.max(0, box.y - ey);
        box.w = Math.min(srcW - box.x, box.w + ex * 2);
        box.h = Math.min(srcH - box.y, box.h + ey * 2);
        boxes.push(box);
        if (boxes.length >= 80) {
            break;
        }
    }
    // 合并相近/重叠框，避免同一行被拆成字符碎片。
    let changed = true;
    while (changed) {
        changed = false;
        for (let i = 0; i < boxes.length; i++) {
            for (let j = i + 1; j < boxes.length; j++) {
                const a = boxes[i];
                const b = boxes[j];
```

```
                const gap = 8;
                const near = a.x - gap <= b.x + b.w && b.x - gap <= a.x + a.w
                    && a.y - gap <= b.y + b.h && b.y - gap <= a.y + a.h;
                if (near) {
                    const x0 = Math.min(a.x, b.x);
                    const y0 = Math.min(a.y, b.y);
                    const x1 = Math.max(a.x + a.w, b.x + b.w);
                    const y1 = Math.max(a.y + a.h, b.y + b.h);
                    a.x = x0;
                    a.y = y0;
                    a.w = x1 - x0;
                    a.h = y1 - y0;
                    boxes.splice(j, 1);
                    changed = true;
                    break;
                }
            }
            if (changed) {
                break;
            }
        }
    }
    return boxes;
}
private recognizeBox(page: PagePixels, box: CandidateBox): string {
    const pad = 2;
    const x0 = Math.max(0, box.x - pad);
    const y0 = Math.max(0, box.y - pad);
    const x1 = Math.min(page.width, box.x + box.w + pad);
    const y1 = Math.min(page.height, box.y + box.h + pad);
    const cropW = Math.max(1, x1 - x0);
    const cropH = Math.max(1, y1 - y0);
    let recW = Math.ceil(REC_HEIGHT * cropW / cropH);
    recW = Math.max(32, Math.ceil(recW / 32) * 32);
    recW = Math.min(recW, 1280);
    const input = new Float32Array(3 * REC_HEIGHT * recW);
    for (let y = 0; y < REC_HEIGHT; y++) {
        const sy = y0 + (y + 0.5) * cropH / REC_HEIGHT - 0.5;
        for (let x = 0; x < recW; x++) {
            if (x >= Math.ceil(REC_HEIGHT * cropW / cropH)) {
                continue;
            }
            const sx = x0 + (x + 0.5) * cropW / Math.ceil(REC_HEIGHT * cropW / cropH) - 0.5;
            const rgb = this.sample(page, sx, sy);
            const base = y * recW + x;
            input[0 * REC_HEIGHT * recW + base] = (rgb[0] / 255 - 0.5) / 0.5;
            input[1 * REC_HEIGHT * recW + base] = (rgb[1] / 255 - 0.5) / 0.5;
            input[2 * REC_HEIGHT * recW + base] = (rgb[2] / 255 - 0.5) / 0.5;
        }
    }
```

```
        const recBuffer = new ArrayBuffer(input.length * 4);
        new Float32Array(recBuffer).set(input);
        const recShape: number[] = [1, 3, REC_HEIGHT, recW];
        const tensor = new OnnxTensor(recBuffer, recShape, 0);
        let result: OnnxResult | undefined = undefined;
        try {
            const inputs: Record<string, OnnxValue> = {};
            inputs['x'] = tensor;
            result = this.recSession!.Run(inputs);
            const outName = this.recSession!.GetOutputNames()[0];
            const outValue = result.GetValueByName(outName);
            if (outValue.GetType() !== 1) {
                return '';
            }
            const outTensor = outValue as OnnxTensor;
            const info: TensorInfo = outTensor.GetInfo();
            const shape: number[] = info.GetShape();
            const seqLen = shape[shape.length - 2];
            const classes = shape[shape.length - 1];
            const data = new Float32Array(outTensor.GetDataAsArrayBuffer());
            let text = '';
            let previous = -1;
            for (let t = 0; t < seqLen; t++) {
                const offset = t * classes;
                let best = 0;
                let prob = data[offset];
                for (let c = 1; c < classes; c++) {
                    if (data[offset + c] > prob) {
                        best = c;
                        prob = data[offset + c];
                    }
                }
                if (best > 0 && best !== previous && best < this.dict.length) {
                    text += this.dict[best];
                }
                previous = best;
            }
            return text;
        }
        finally {
            tensor.Close();
            if (result !== undefined) {
                result.Close();
            }
        }
    }
    private resizeToCHW(page: PagePixels, dstW: number, dstH: number, out: Float32Array): void {
        const plane = dstW * dstH;
        for (let y = 0; y < dstH; y++) {
            const sy = (y + 0.5) * page.height / dstH - 0.5;
```

```
            for (let x = 0; x < dstW; x++) {
                const sx = (x + 0.5) * page.width / dstW - 0.5;
                const rgb = this.sample(page, sx, sy);
                const idx = y * dstW + x;
                out[idx] = (rgb[0] / 255 - 0.5) / 0.5;
                out[plane + idx] = (rgb[1] / 255 - 0.5) / 0.5;
                out[plane * 2 + idx] = (rgb[2] / 255 - 0.5) / 0.5;
            }
        }
    }
    private sample(page: PagePixels, fx: number, fy: number): Array<number> {
        const x = Math.max(0, Math.min(page.width - 1, fx));
        const y = Math.max(0, Math.min(page.height - 1, fy));
        const x0 = Math.floor(x);
        const y0 = Math.floor(y);
        const x1 = Math.min(page.width - 1, x0 + 1);
        const y1 = Math.min(page.height - 1, y0 + 1);
        const wx = x - x0;
        const wy = y - y0;
        const i00 = (y0 * page.width + x0) * 4;
        const i10 = (y0 * page.width + x1) * 4;
        const i01 = (y1 * page.width + x0) * 4;
        const i11 = (y1 * page.width + x1) * 4;
        const out: Array<number> = [0, 0, 0];
        for (let c = 0; c < 3; c++) {
            const top = page.rgba[i00 + c] * (1 - wx) + page.rgba[i10 + c] * wx;
            const bottom = page.rgba[i01 + c] * (1 - wx) + page.rgba[i11 + c] * wx;
            out[c] = top * (1 - wy) + bottom * wy;
        }
        return out;
    }
}
// ==================== 文件: corerender/src/main/ets/OverlayRenderer.ets ====================
import { DisplayRect } from './Geometry';
import { LayoutEngine, OverlayRegion, RenderOptions, TextLayout } from './LayoutEngine';
import { OverlayItem } from './PageOverlay';

type Ctx2D = CanvasRenderingContext2D | OffscreenCanvasRenderingContext2D;

/** 覆盖层配色（白块底 + 文字色）；反色模式由调用方传入深底浅字。 */
export class OverlayColors {
  boxFill: string = '#FFFFFF';
  textColor: string = '#1A1A1A';

  static make(boxFill: string, textColor: string): OverlayColors {
    const colors = new OverlayColors();
    colors.boxFill = boxFill;
    colors.textColor = textColor;
    return colors;
  }
```

```
}

/**
 * Overlay painter.
 *
 * 坐标约定（QA-1 修复的关键）：
 * - LayoutRun.x/y 是**页面坐标系**下字格中心的绝对坐标（由 LayoutEngine 算出）；
 * - 屏幕绘制走 paintPlan：逐字做 x' = dx + x*scale、y' = dy + y*scale、
 *   fontSize' = fontSize*scale，从而在目标分辨率上直接光栅化（文字清晰，不会被位图缩放糊掉）；
 * - 导出离屏合成直接传 scale=1/dx=0/dy=0，像素坐标 1:1。
 *
 * ⚠ 字体单位坑（本轮 AUTO 字号偏小/字距巨大的根因）：
 * ArkUI Canvas 的**绘制坐标默认单位是 vp**，但 ctx.font 中不带单位的字号按 **px（物理像素）**
 * 解释（官方 FAQ 示例写作 'bold 18vp sans-serif' 即为此意）。若在 vp 画布上写 '21px'，
 * 实际只有 21 物理 px ≈ 6vp，而逐字坐标仍是 vp → 表现为“字号小了 density 倍、间距显得巨大”。
 * 因此 paintPlan/drawText 需要显式指定字号单位：
 * - 屏幕 Canvas（默认 vp 单位）→ fontUnit='vp'；
 * - 导出 OffscreenCanvas（用 LengthMetricsUnit.PX 创建）→ fontUnit='px'。
 */
export class OverlayRenderer {
  /**
   * 组装 canvas font 字符串。
   * ArkUI Canvas 的 ctx.font 字号按 **物理 px** 解释（坐标则是 vp），
   * 所以屏幕路径要把 vp 字号换算成 px；导出 PX 画布本来就是 px，直接使用。
   */
  static fontOf(fontSize: number, fontUnit: string): string {
    // 内置 Noto Sans CJK SC，由 Index.aboutToAppear 注册为 MoyiSans。
    if (fontUnit === 'px') {
      return fontSize + 'px MoyiSans';
    }
    return vp2px(fontSize) + 'px MoyiSans';
  }

  /** White base block covering the bubble area (display/page coords). */
  static drawBox(ctx: Ctx2D, box: DisplayRect, fill: string = '#FFFFFF'): void {
    ctx.fillStyle = fill;
    ctx.fillRect(box.x, box.y, box.w, box.h);
  }

  /** Draw already-laid-out characters (runs use the same coordinate space as the canvas). */
  static drawText(ctx: Ctx2D, layout: TextLayout, color: string = '#1A1A1A',
    fontUnit: string = 'vp'): void {
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    ctx.fillStyle = color;
    let lastFs = -1;
    for (let i = 0; i < layout.runs.length; i++) {
      const run = layout.runs[i];
      if (Math.abs(run.fontSize - lastFs) > 0.01) {
        ctx.font = OverlayRenderer.fontOf(run.fontSize, fontUnit);
```

```
        lastFs = run.fontSize;
      }
      ctx.fillText(run.char, run.x, run.y);
    }
  }

  /** White block + text in one call (legacy display-space path, vp canvas). */
  static drawRegion(ctx: Ctx2D, region: OverlayRegion, layout: TextLayout,
    color: string = '#1A1A1A', boxFill: string = '#FFFFFF', fontUnit: string = 'vp'): void {
    OverlayRenderer.drawBox(ctx, region.box, boxFill);
    OverlayRenderer.drawText(ctx, layout, color, fontUnit);
  }

  /** Layout + draw a bubble's text with the given render options (legacy path, vp canvas). */
  static renderRegionText(ctx: Ctx2D, text: string, region: OverlayRegion,
    opts: RenderOptions, color: string = '#1A1A1A', boxFill: string = '#FFFFFF',
    fontUnit: string = 'vp'): TextLayout {
    const layout = LayoutEngine.layout(text, region.box, opts);
    OverlayRenderer.drawRegion(ctx, region, layout, color, boxFill, fontUnit);
    return layout;
  }

  /** 页面坐标 → 显示坐标（contain-fit）。 */
  static toDisplayRect(box: DisplayRect, scale: number, dx: number, dy: number): DisplayRect {
    const r: DisplayRect = {
      x: dx + box.x * scale,
      y: dy + box.y * scale,
      w: box.w * scale,
      h: box.h * scale
    };
    return r;
  }

  /**
   * 绘制页面坐标系排版结果：逐字变换到显示坐标后直接光栅化。
   * @param scale contain-fit 比例（vp/页面px 或 px/页面px）
   * @param dx/@param dy 居中偏移
   * @param fontUnit 字号单位：vp 画布用 'vp'，PX 画布（导出）用 'px'
   */
  static paintPlan(ctx: Ctx2D, items: Array<OverlayItem>, scale: number, dx: number, dy: number,
    colors: OverlayColors, fontUnit: string = 'vp'): void {
    if (scale <= 0 || items.length === 0) {
      return;
    }
    ctx.textAlign = 'center';
    ctx.textBaseline = 'middle';
    for (let i = 0; i < items.length; i++) {
      const item = items[i];
      const box = item.box;
      if (colors.boxFill !== 'none') {
```

```
        ctx.fillStyle = colors.boxFill;
        ctx.fillRect(dx + box.x * scale, dy + box.y * scale, box.w * scale, box.h * scale);
      }
      const itemColor = item.textColor !== undefined && item.textColor.length > 0
        ? item.textColor : colors.textColor;
      const outline = itemColor.toUpperCase() === '#1A1A1A' ? '#FFFFFF' : '#1A1A1A';
      ctx.fillStyle = itemColor;
      let lastFs = -1;
      const runs = item.layout.runs;
      for (let k = 0; k < runs.length; k++) {
        const run = runs[k];
        const fs = run.fontSize * scale;
        if (fs <= 0) {
          continue;
        }
        if (Math.abs(fs - lastFs) > 0.01) {
          ctx.font = OverlayRenderer.fontOf(fs, fontUnit);
          lastFs = fs;
        }
        // 1px 级描边保证浅底黑字/深底白字都能读清。
        ctx.lineWidth = Math.max(1, fs * 0.06);
        ctx.lineJoin = 'round';
        ctx.strokeStyle = outline;
        ctx.strokeText(run.char, dx + run.x * scale, dy + run.y * scale);
        ctx.fillText(run.char, dx + run.x * scale, dy + run.y * scale);
      }
    }
  }

  /**
   * 单气泡便捷入口：在页面坐标排版后按 fit 变换绘制。
   * 返回排版结果（供调试/自检查看 measuredFontSize 等）。
   */
  static paintFitted(ctx: Ctx2D, text: string, box: DisplayRect, opts: RenderOptions,
    scale: number, dx: number, dy: number, colors: OverlayColors,
    fontUnit: string = 'vp'): TextLayout {
    const layout = LayoutEngine.layout(text, box, opts);
    const item: OverlayItem = { box: box, text: text, layout: layout, merged: 1 };
    const items: Array<OverlayItem> = [item];
    OverlayRenderer.paintPlan(ctx, items, scale, dx, dy, colors, fontUnit);
    return layout;
  }

  /** 绘制一个 OCR 框的版面（页面坐标排版 + fit 变换）。 */
  static paintPageRegions(ctx: Ctx2D, text: string, pageBox: DisplayRect, opts: RenderOptions,
    scale: number, dx: number, dy: number, colors: OverlayColors,
    fontUnit: string = 'vp'): TextLayout {
    return OverlayRenderer.paintFitted(ctx, text, pageBox, opts, scale, dx, dy, colors,
fontUnit);
  }
```

```
}
// ==================== 文件: corerender/src/main/ets/LayoutEngine.ets ====================
import { DisplayRect } from './Geometry';

/** Text orientation inside bubbles. */
export enum RenderOrientation {
  HORIZONTAL = 0,
  VERTICAL = 1,
  /** 按检测框宽高自动选择横排/竖排（覆盖层逐框推断）。 */
  AUTO = 2
}

/** Column flow direction for vertical text (manga default: RTL). */
export enum VerticalDirection {
  RTL = 0,
  LTR = 1
}

export enum FontSizeMode {
  AUTO = 0,
  FIXED = 1
}

/**
 * 全局渲染设置（preferences starflow_reader 持久化）。
 * lineSpacing / charSpacing 为可选增量（×字号），0 表示自然间距；
 * 作为可选字段保证既有调用方的对象字面量继续兼容。
 */
export interface RenderOptions {
  orientation: RenderOrientation;
  verticalDirection: VerticalDirection;
  fontSizeMode: FontSizeMode;
  fixedFontSize: number;
  padding: number;
  /** 额外行距（横排行距 / 竖排列距），单位 = ×字号，0 为自然间距。 */
  lineSpacing?: number;
  /** 额外字距（横排字距 / 竖排列内字距），单位 = ×字号，0 为自然间距。 */
  charSpacing?: number;
  /** 页面像素宽（可选）：用于把 OCR 框裁剪到页面内，避免白块越界。 */
  pageWidth?: number;
  /** 页面像素高（可选）。 */
  pageHeight?: number;
}

/** One positioned character; x/y mark the char cell CENTER (display/page coords). */
export interface LayoutRun {
  char: string;
  x: number;
  y: number;
  fontSize: number;
```

```
}

export interface TextLayout {
  runs: Array<LayoutRun>;
  truncated: boolean;
  measuredFontSize: number;
}

/** A bubble region; box is in PAGE pixel coordinates (renderer applies the fit transform). */
export interface OverlayRegion {
  box: DisplayRect;
  original: string;
  translated: string;
  /** 可选文字色（由去字底图采样得到）；空/未设时由渲染端默认色兜底。 */
  textColor?: string;
}

/** Character width measurer; default approximation is deterministic for tests. */
export interface CharMeasurer {
  measure(char: string, fontSize: number): number;
}

/**
 * 竖排列式回归自检（单测式，纯函数、无 UI 依赖）。
 * 供阅读器设置面板 / 开发者工作台直接调用展示；不通过时 detail 给出实测值。
 */
export class SelfTestResult {
  name: string = '';
  passed: boolean = false;
  detail: string = '';
}

export function makeSelfTestResult(name: string, passed: boolean, detail: string):
SelfTestResult {
  const r: SelfTestResult = { name: name, passed: passed, detail: detail };
  return r;
}

const MIN_FONT_AUTO = 1;
const MIN_FONT_FIXED = 6;
const MAX_FONT = 72;
/** 横排自然行距系数。 */
const H_LINE_RATIO = 1.3;
/** 竖排列内自然字步进系数。 */
const V_CHAR_RATIO = 1.08;
/** 竖排列间自然步进系数。 */
const V_COL_RATIO = 1.1;
/** 竖排列距（列间空白）拉伸上限（×字号）：既能“列距拉伸铺满”，又不会散架。 */
const MAX_COL_GAP = 2.5;
const EPS = 0.01;
```

```
const ELLIPSIS = '…';
const NO_START_CHARS = '，。、！？；：…）」』】〉》,.!?;:)';

/** 汉字/全角统一按 1 字宽，拉丁按比例；确定性便于单测。 */
export class ApproxCharMeasurer implements CharMeasurer {
  measure(char: string, fontSize: number): number {
    if (char === ' ') {
      return fontSize * 0.3;
    }
    const code = char.charCodeAt(0);
    if (code < 0x80) {
      const isAlnum = (code >= 0x30 && code <= 0x39) || (code >= 0x41 && code <= 0x5A) || (code
>= 0x61 && code <= 0x7A);
      return isAlnum ? fontSize * 0.56 : fontSize * 0.5;
    }
    return fontSize;
  }
}

export function defaultRenderOptions(): RenderOptions {
  const o: RenderOptions = {
    orientation: RenderOrientation.VERTICAL,
    verticalDirection: VerticalDirection.RTL,
    fontSizeMode: FontSizeMode.AUTO,
    fixedFontSize: 16,
    padding: 4,
    lineSpacing: 0,
    charSpacing: 0
  };
  return o;
}

export function makeOverlayRegion(box: DisplayRect, original: string, translated: string):
OverlayRegion {
  const r: OverlayRegion = { box: box, original: original, translated: translated };
  return r;
}

function emptyLayout(): TextLayout {
  const t: TextLayout = { runs: [], truncated: false, measuredFontSize: 0 };
  return t;
}

function isNoStart(c: string): boolean {
  return NO_START_CHARS.indexOf(c) >= 0;
}

/** 点串成员（用于“点串绝不跨行/跨列”规则）。 */
function isEllipsisDot(c: string): boolean {
  return c === '…' || c === '.' || c === '。';
```

```
}

/** 拆成字符数组（保留代理对，去掉换行；竖排里换行没有意义）。 */
function splitChars(text: string): Array<string> {
  const chars: Array<string> = [];
  let i = 0;
  while (i < text.length) {
    const code = text.charCodeAt(i);
    if (code >= 0xD800 && code <= 0xDBFF && i + 1 < text.length) {
      const next = text.charCodeAt(i + 1);
      if (next >= 0xDC00 && next <= 0xDFFF) {
        chars.push(text.substring(i, i + 2));
        i += 2;
        continue;
      }
    }
    const ch = text.charAt(i);
    if (ch !== '\n' && ch !== '\r') {
      chars.push(ch);
    }
    i++;
  }
  return chars;
}

function resolvePadding(box: DisplayRect, padding: number): number {
  const p = padding > 0 ? padding : 0;
  const cap = Math.min(box.w, box.h) / 4;
  return p < cap ? p : cap;
}

function spacingOf(raw: number | undefined): number {
  if (raw === undefined) {
    return 0;
  }
  return raw > 0 ? raw : 0;
}

/** 复制 RenderOptions 并覆盖间距（用于“保底不丢字”：放弃额外间距再排一次）。 */
function optionsWithSpacing(opts: RenderOptions, lineSpacing: number, charSpacing: number):
RenderOptions {
  const o: RenderOptions = {
    orientation: opts.orientation,
    verticalDirection: opts.verticalDirection,
    fontSizeMode: opts.fontSizeMode,
    fixedFontSize: opts.fixedFontSize,
    padding: opts.padding,
    lineSpacing: lineSpacing,
    charSpacing: charSpacing
  };
```

```
  return o;
}

// ---------------- 横排 ----------------

/**
 * 横排贪心换行（含字距预算）：
 * k 个字的行实际占 sum(width) + (k-1)*tracking，断行必须把 tracking 算进去。
 */
function wrapLines(chars: Array<string>, m: CharMeasurer, fs: number, maxWidth: number,
  tracking: number): Array<Array<string>> {
  const lines: Array<Array<string>> = [];
  let cur: Array<string> = [];
  let curW = 0;
  for (let idx = 0; idx < chars.length; idx++) {
    const c = chars[idx];
    const w = m.measure(c, fs);
    const addW = cur.length > 0 ? w + tracking : w;
    if (cur.length > 0 && curW + addW > maxWidth + EPS) {
      const lastChar = cur[cur.length - 1];
      const lastW = m.measure(lastChar, fs);
      if (isNoStart(c) && cur.length >= 2 && lastW + w + tracking <= maxWidth + EPS) {
        cur.pop();
        lines.push(cur);
        cur = [lastChar];
        curW = lastW;
      } else {
        lines.push(cur);
        cur = [];
        curW = 0;
      }
    }
    if (cur.length > 0) {
      curW += w + tracking;
    } else {
      curW = w;
    }
    cur.push(c);
  }
  if (cur.length > 0) {
    lines.push(cur);
  }
  return lines;
}

/** 由已断好的行生成 runs：整块垂直居中、行内水平居中；字距/行距不做额外拉伸。 */
function buildHorizontalRuns(lines: Array<Array<string>>, m: CharMeasurer, box: DisplayRect,
  padding: number, fs: number, opts: RenderOptions): Array<LayoutRun> {
  const runs: Array<LayoutRun> = [];
  const bw = box.w - padding * 2;
```

```
  const bh = box.h - padding * 2;
  const tracking = spacingOf(opts.charSpacing) * fs;
  // 行距只由用户设置决定（自然 1.3 + 额外行距），不为铺满额外拉伸；剩余空白整块居中。
  const lead = fs * (H_LINE_RATIO + spacingOf(opts.lineSpacing));
  const n = lines.length;
  const blockH = (n - 1) * lead + fs;
  const vOff = blockH < bh ? (bh - blockH) / 2 : 0;
  for (let i = 0; i < n; i++) {
    const line = lines[i];
    let lw = 0;
    for (let k = 0; k < line.length; k++) {
      lw += m.measure(line[k], fs);
    }
    if (line.length > 1) {
      lw += tracking * (line.length - 1);
    }
    let xCur = lw < bw ? (bw - lw) / 2 : 0;
    const yCenter = vOff + fs / 2 + i * lead;
    for (let k = 0; k < line.length; k++) {
      const c = line[k];
      const w = m.measure(c, fs);
      const run: LayoutRun = {
        char: c,
        x: box.x + padding + xCur + w / 2,
        y: box.y + padding + yCenter,
        fontSize: fs
      };
      runs.push(run);
      xCur += w + tracking;
    }
  }
  return runs;
}

/** 横排试排；装不下返回 undefined（不做截断）。 */
function tryHorizontal(chars: Array<string>, m: CharMeasurer, box: DisplayRect,
  padding: number, fs: number, opts: RenderOptions): TextLayout | undefined {
  const bw = box.w - padding * 2;
  const bh = box.h - padding * 2;
  if (bw <= 0 || bh <= 0) {
    return undefined;
  }
  const tracking = spacingOf(opts.charSpacing) * fs;
  const lines = wrapLines(chars, m, fs, bw, tracking);
  if (lines.length === 0) {
    return undefined;
  }
  const lead = fs * (H_LINE_RATIO + spacingOf(opts.lineSpacing));
  const blockH = (lines.length - 1) * lead + fs;
  if (blockH > bh + EPS) {
```

```
    return undefined;
  }
  const runs = buildHorizontalRuns(lines, m, box, padding, fs, opts);
  const t: TextLayout = { runs: runs, truncated: false, measuredFontSize: fs };
  return t;
}

// ---------------- 竖排 ----------------

/** 竖排每列可容纳字符数：末字下边缘必须落在可用高内。 */
function verticalCapacity(bh: number, fs: number, charStep: number): number {
  if (charStep <= 0 || bh < fs) {
    return 0;
  }
  return Math.floor((bh - fs) / charStep) + 1;
}

/**
 * 按列容量切分，列内自上而下逐字（顺序即 runs 顺序）；
 * 切点绝不落在点串内部（三个点的省略号整串进下一列）。
 */
function wrapColumns(chars: Array<string>, cap: number): Array<Array<string>> {
  const cols: Array<Array<string>> = [];
  if (cap < 1) {
    return cols;
  }
  let i = 0;
  while (i < chars.length) {
    let end = i + cap < chars.length ? i + cap : chars.length;
    if (end < chars.length && isEllipsisDot(chars[end - 1]) && isEllipsisDot(chars[end])) {
      let runStart = end - 1;
      while (runStart > i && isEllipsisDot(chars[runStart - 1])) {
        runStart--;
      }
      if (runStart > i) {
        end = runStart;
      }
    }
    const col: Array<string> = [];
    for (let k = i; k < end; k++) {
      col.push(chars[k]);
    }
    cols.push(col);
    i = end;
  }
  return cols;
}

/**
 * 竖排试排（页面坐标）：列从右到左（RTL）或左到右（LTR），列内自上而下；
```

```
    if (data.trim() === '[DONE]') {
      this.callbacks.onDone();
      return;
    }
    this.callbacks.onEvent(data);
  }
}
// ==================== 文件: commons/Index.ets ====================
/**
 * commons HAR entry.
 */
export { ThemeTokens } from './src/main/ets/ThemeTokens';
export { Log } from './src/main/ets/Log';
export { naturalSort, naturalCompare } from './src/main/ets/SortUtil';
export {
  WindowSizeClass,
  DeviceClass,
  sizeClassOfWidth,
  currentDeviceClass,
  screenWidthVp,
  startBreakpointObserver,
  observeBreakpoint
} from './src/main/ets/Breakpoint';export { toMediaUri, isUri } from './src/main/ets/PathUtil';
// ==================== 文件: commons/src/main/ets/Breakpoint.ets ====================
import { display, mediaquery, window } from '@kit.ArkUI';
import { deviceInfo } from '@kit.BasicServicesKit';
import { common } from '@kit.AbilityKit';

/**
 * One-time-development, multi-device sizing (一次开发，多端部署).
 * Official breakpoints: sm < 600vp, md 600–839vp, lg >= 840vp.
 */
export enum WindowSizeClass {
  SM = 0,
  MD = 1,
  LG = 2
}

/** Coarse hardware class used to tailor interaction (hover cursor etc.). */
export enum DeviceClass {
  PHONE = 0,
  TABLET = 1,
  PC = 2,
  OTHER = 3
}

export function sizeClassOfWidth(vp: number): WindowSizeClass {
  if (vp >= 840) {
    return WindowSizeClass.LG;
  }
```

```
  if (vp >= 600) {
    return WindowSizeClass.MD;
  }
  return WindowSizeClass.SM;
}

export function currentDeviceClass(): DeviceClass {
  const t = deviceInfo.deviceType;
  if (t === '2in1' || t === 'pc') {
    return DeviceClass.PC;
  }
  if (t === 'tablet') {
    return DeviceClass.TABLET;
  }
  if (t === 'phone' || t === 'default') {
    return DeviceClass.PHONE;
  }
  return DeviceClass.OTHER;
}

/** Screen width in vp (fallback / initial value before layout). */
export function screenWidthVp(): number {
  const d = display.getDefaultDisplaySync();
  const density = d.densityPixels > 0 ? d.densityPixels : 1;
  return d.width / density;
}

function vpWidthOfScreen(): number {
  return screenWidthVp();
}

/**
 * Reliable breakpoint observer for the app shell:
 * initial value is derived from the real display/window size (media queries can
 * report a pre-layout width on tablets), then updated on window resize.
 * Must be called from a UIAbility/UI page context.
 */
export async function startBreakpointObserver(context: common.Context,
  onChange: (c: WindowSizeClass) => void): Promise<() => void> {
  let win: window.Window | undefined = undefined;
  try {
    win = await window.getLastWindow(context);
  } catch (e) {
    win = undefined;
  }
  const apply = () => {
    let vp = vpWidthOfScreen();
    if (win !== undefined) {
      try {
        const rect = win.getWindowProperties().windowRect;
```

```
        const density = display.getDefaultDisplaySync().densityPixels;
        if (rect.width > 0 && density > 0) {
          vp = rect.width / density;
        }
      } catch (e) {
        // keep screen-derived fallback
      }
    }
    onChange(sizeClassOfWidth(vp));
  };
  apply();
  const sizeCb = (size: window.Size) => {
    const density = display.getDefaultDisplaySync().densityPixels;
    const vp = density > 0 ? size.width / density : size.width;
    onChange(sizeClassOfWidth(vp));
  };
  if (win !== undefined) {
    try {
      win.on('windowSizeChange', sizeCb);
    } catch (e) {
      // ignore and rely on media queries below
    }
  }
  const smListener = mediaquery.matchMediaSync('(width>=600vp)');
  const lgListener = mediaquery.matchMediaSync('(width>=840vp)');
  const emit = () => {
    let c = WindowSizeClass.SM;
    if (lgListener.matches) {
      c = WindowSizeClass.LG;
    } else if (smListener.matches) {
      c = WindowSizeClass.MD;
    }
    onChange(c);
  };
  smListener.on('change', emit);
  lgListener.on('change', emit);
  return () => {
    if (win !== undefined) {
      try {
        win.off('windowSizeChange', sizeCb);
      } catch (e) {
        // ignore
      }
    }
    smListener.off('change', emit);
    lgListener.off('change', emit);
  };
}

/**
```

```
 * Lightweight media-query observer (page level). Returns a stop function.
 */
export function observeBreakpoint(onChange: (c: WindowSizeClass) => void): () => void {
  const smListener = mediaquery.matchMediaSync('(width>=600vp)');
  const lgListener = mediaquery.matchMediaSync('(width>=840vp)');
  const emit = () => {
    let c = WindowSizeClass.SM;
    if (lgListener.matches) {
      c = WindowSizeClass.LG;
    } else if (smListener.matches) {
      c = WindowSizeClass.MD;
    }
    onChange(c);
  };
  smListener.on('change', emit);
  lgListener.on('change', emit);
  emit();
  return () => {
    smListener.off('change', emit);
    lgListener.off('change', emit);
  };
}
// ==================== 文件: commons/src/main/ets/Log.ets ====================
import { hilog } from '@kit.PerformanceAnalysisKit';

/**
 * Thin hilog wrapper with a fixed domain and per-call tags.
 */
export class Log {
  private static readonly DOMAIN: number = 0x0301;

  static debug(tag: string, msg: string): void {
    hilog.debug(Log.DOMAIN, tag, '%{public}s', msg);
  }

  static info(tag: string, msg: string): void {
    hilog.info(Log.DOMAIN, tag, '%{public}s', msg);
  }

  static warn(tag: string, msg: string): void {
    hilog.warn(Log.DOMAIN, tag, '%{public}s', msg);
  }

  static error(tag: string, msg: string): void {
    hilog.error(Log.DOMAIN, tag, '%{public}s', msg);
  }
}
// ==================== 文件: commons/src/main/ets/PathUtil.ets ====================
import { fileUri } from '@kit.CoreFileKit';
```

```
/**
 * Media path helpers.
 *
 * ArkUI's Image component does NOT accept a raw sandbox path like
 * /data/storage/el2/base/haps/entry/files/xxx.png — it needs a sandbox URI
 * (file://<bundle>/<sandboxPath>) or a PixelMap. Imported pages, covers and
 * exported previews are stored as sandbox paths, so every Image source and
 * ImageSource creation should pass through toMediaUri().
 */
export function toMediaUri(pathOrUri: string): string {
  if (pathOrUri.length === 0) {
    return pathOrUri;
  }
  if (pathOrUri.startsWith('file://') || pathOrUri.startsWith('http://') ||
    pathOrUri.startsWith('https://') || pathOrUri.startsWith('datashare://') ||
    pathOrUri.startsWith('content://') || pathOrUri.startsWith('ohos://') ||
    pathOrUri.startsWith('$')) {
    return pathOrUri;
  }
  if (pathOrUri.startsWith('/')) {
    try {
      return fileUri.getUriFromPath(pathOrUri);
    } catch (e) {
      return pathOrUri;
    }
  }
  return pathOrUri;
}

/** True when the value is already a URI accepted by Image / ImageSource. */
export function isUri(value: string): boolean {
  return value.startsWith('file://') || value.startsWith('http://') ||
value.startsWith('https://') ||
    value.startsWith('datashare://') || value.startsWith('content://');
}
// ==================== 文件: commons/src/main/ets/SortUtil.ets ====================
/**
 * Digit-aware natural ordering so page names sort v2 < v10 (plan D10).
 */
function isDigit(c: string): boolean {
  return c >= '0' && c <= '9';
}

/**
 * Compare two strings naturally: digit runs compare numerically,
 * other characters compare by code point. Returns <0 / 0 / >0.
 */
export function naturalCompare(a: string, b: string): number {
  let i = 0;
  let j = 0;
```

```
  while (i < a.length && j < b.length) {
    const ca = a.charAt(i);
    const cb = b.charAt(j);
    if (isDigit(ca) && isDigit(cb)) {
      let ai = i;
      let bj = j;
      while (ai < a.length && isDigit(a.charAt(ai))) {
        ai++;
      }
      while (bj < b.length && isDigit(b.charAt(bj))) {
        bj++;
      }
      const rawA = a.substring(i, ai);
      const rawB = b.substring(j, bj);
      let za = 0;
      while (za < rawA.length - 1 && rawA.charAt(za) === '0') {
        za++;
      }
      let zb = 0;
      while (zb < rawB.length - 1 && rawB.charAt(zb) === '0') {
        zb++;
      }
      const tA = rawA.substring(za);
      const tB = rawB.substring(zb);
      if (tA.length !== tB.length) {
        return tA.length - tB.length;
      }
      if (tA !== tB) {
        return tA < tB ? -1 : 1;
      }
      if (rawA.length !== rawB.length) {
        return rawA.length - rawB.length;
      }
      i = ai;
      j = bj;
    } else {
      const codeA = a.codePointAt(i) ?? 0;
      const codeB = b.codePointAt(j) ?? 0;
      if (codeA !== codeB) {
        return codeA - codeB;
      }
      i++;
      j++;
    }
  }
  const remA = a.length - i;
  const remB = b.length - j;
  if (remA === remB) {
    return 0;
  }
```

```
  return remA < remB ? -1 : 1;
}

/** Return a NEW naturally-sorted array (input untouched). */
export function naturalSort(names: Array<string>): Array<string> {
  const copy: Array<string> = names.slice(0, names.length);
  copy.sort((a: string, b: string): number => naturalCompare(a, b));
  return copy;
}
// ==================== 文件: commons/src/main/ets/ThemeTokens.ets ====================
/**
 * Moyi HarmonyOS design tokens — 蓝白 / 蓝黑（默认蓝白）.
 * 已移除紫色主题；token 名称保持不变，页面无需改动。
 */
export type ThemeMode = 'blue' | 'blueDark';

export class ThemeTokens {
  private static modeValue: ThemeMode = 'blue';

  static SURFACE: string = '#FFFFFF';
  static SURFACE_VARIANT: string = '#F2F7FF';
  static SURFACE_SECONDARY: string = '#E9F2FF';
  static SURFACE_HIGH: string = '#DCE9FB';
  static CARD_BG: string = '#FFFFFF';
  static CARD_HOVER: string = '#F5FAFF';
  static DIVIDER: string = '#DCE8F7';
  static SCRIM: string = '#CC0B1220';

  static TEXT_PRIMARY: string = '#0B1F33';
  static TEXT_SECONDARY: string = '#3E566F';
  static TEXT_HINT: string = '#6C8299';
  static TEXT_TERTIARY: string = '#93A7BC';
  static TEXT_ON_PRIMARY: string = '#FFFFFF';

  static PRIMARY: string = '#0A66FF';
  static PRIMARY_DEEP: string = '#004FCC';
  static PRIMARY_SOFT: string = '#5B97FF';
  static ACCENT: string = '#2F7CFF';
  static RED: string = '#D93B4F';
  static SUCCESS: string = '#12805C';
  static WARN: string = '#B26A00';
  static WHITE: string = '#FFFFFF';
  static BLACK: string = '#0A1220';

  static MATERIAL_TINT: string = '#CCFFFFFF';
  static GLASS_BORDER: string = '#661F6FEB';

  static readonly READER_BG_BLACK: string = '#000000';
  static readonly READER_BG_NOCTURNE: string = '#0B0813';
  static readonly READER_BG_PLUM: string = '#16112A';
```

```
  static readonly READER_BG_DEEP: string = '#221A38';
  static readonly READER_BG_PAPER: string = '#F5EFE4';
  static readonly READER_BG_WHITE: string = '#FFFFFF';
  static readonly READER_BG_GRAY: string = '#808080';

  static readonly FONT_TITLE: number = 20;
  static readonly FONT_SUBTITLE: number = 17;
  static readonly FONT_BODY: number = 16;
  static readonly FONT_CAPTION: number = 13;
  static readonly FONT_SMALL: number = 12;
  static readonly FONT_DISPLAY: number = 28;

  static readonly RADIUS_CARD: number = 16;
  static readonly RADIUS_SMALL: number = 10;
  static readonly RADIUS_PILL: number = 999;

  static readonly PAGE_PADDING: number = 16;
  static readonly CARD_PADDING: number = 14;
  static readonly GAP: number = 12;

  static readonly DURATION_FAST: number = 150;
  static readonly DURATION_BASE: number = 240;

  /** 蓝白/蓝黑运行时切换；未知旧值（purple/oled/system）迁移到蓝白。 */
  static applyMode(mode: string): void {
    if (mode === 'blueDark') {
      ThemeTokens.modeValue = 'blueDark';
      ThemeTokens.SURFACE = '#071426';
      ThemeTokens.SURFACE_VARIANT = '#0D1F35';
      ThemeTokens.SURFACE_SECONDARY = '#12283F';
      ThemeTokens.SURFACE_HIGH = '#18334F';
      ThemeTokens.CARD_BG = '#0D1F35';
      ThemeTokens.CARD_HOVER = '#14304A';
      ThemeTokens.DIVIDER = '#1E3A57';
      ThemeTokens.SCRIM = '#E6000000';
      ThemeTokens.TEXT_PRIMARY = '#E8F2FF';
      ThemeTokens.TEXT_SECONDARY = '#A9C2DD';
      ThemeTokens.TEXT_HINT = '#7693B2';
      ThemeTokens.TEXT_TERTIARY = '#5A7693';
      ThemeTokens.TEXT_ON_PRIMARY = '#FFFFFF';
      ThemeTokens.PRIMARY = '#5B9CFF';
      ThemeTokens.PRIMARY_DEEP = '#3B7BDD';
      ThemeTokens.PRIMARY_SOFT = '#8DBCFF';
      ThemeTokens.ACCENT = '#6EA8FF';
      ThemeTokens.RED = '#FF7A8A';
      ThemeTokens.SUCCESS = '#4FD1A5';
      ThemeTokens.WARN = '#F0B95C';
      ThemeTokens.WHITE = '#FFFFFF';
      ThemeTokens.BLACK = '#000000';
      ThemeTokens.MATERIAL_TINT = '#33FFFFFF';
```

```
      ThemeTokens.GLASS_BORDER = '#3DFFFFFF';
      return;
    }
    ThemeTokens.modeValue = 'blue';
    ThemeTokens.SURFACE = '#FFFFFF';
    ThemeTokens.SURFACE_VARIANT = '#F2F7FF';
    ThemeTokens.SURFACE_SECONDARY = '#E9F2FF';
    ThemeTokens.SURFACE_HIGH = '#DCE9FB';
    ThemeTokens.CARD_BG = '#FFFFFF';
    ThemeTokens.CARD_HOVER = '#F5FAFF';
    ThemeTokens.DIVIDER = '#DCE8F7';
    ThemeTokens.SCRIM = '#CC0B1220';
    ThemeTokens.TEXT_PRIMARY = '#0B1F33';
    ThemeTokens.TEXT_SECONDARY = '#3E566F';
    ThemeTokens.TEXT_HINT = '#6C8299';
    ThemeTokens.TEXT_TERTIARY = '#93A7BC';
    ThemeTokens.TEXT_ON_PRIMARY = '#FFFFFF';
    ThemeTokens.PRIMARY = '#0A66FF';
    ThemeTokens.PRIMARY_DEEP = '#004FCC';
    ThemeTokens.PRIMARY_SOFT = '#5B97FF';
    ThemeTokens.ACCENT = '#2F7CFF';
    ThemeTokens.RED = '#D93B4F';
    ThemeTokens.SUCCESS = '#12805C';
    ThemeTokens.WARN = '#B26A00';
    ThemeTokens.WHITE = '#FFFFFF';
    ThemeTokens.BLACK = '#0A1220';
    ThemeTokens.MATERIAL_TINT = '#CCFFFFFF';
    ThemeTokens.GLASS_BORDER = '#661F6FEB';
  }

  static mode(): ThemeMode {
    return ThemeTokens.modeValue;
  }

  static isLight(): boolean {
    return ThemeTokens.modeValue === 'blue';
  }
}
// ==================== 文件: entry/src/main/cpp/types/libllamacpp/index.d.ets ====================
/**
 * NAPI contract for libllamacpp.so (plan D11).
 * generate() streams tokens through onToken and reports completion
 * (full text, or error message when isError) through onDone.
 */
export const getVersion: () => string;

export const loadModel: (modelPath: string) => number;

export const generate: (handle: number, prompt: string, maxTokens: number, temperature: number,
  onToken: (token: string) => void,
```

```
  onDone: (text: string, isError: boolean) => void) => void;

export const stopGeneration: (handle: number) => void;

export const unloadModel: (handle: number) => void;
// ==================== 文件: entry/src/main/cpp/types/libocrasync/index.d.ets ====================
export const initRec: (modelPath: string) => boolean;
export const initDet: (modelPath: string) => boolean;
export const detectBoxes: (rgba: ArrayBuffer, width: number, height: number) => Array<number>;
export const recognizeBoxes: (rgba: ArrayBuffer, width: number, height: number,
  boxes: Array<number>, dict: string) => Array<string>;
// ==================== 文件: entry/src/test/List.test.ets ====================
import { describe, it, expect } from '@ohos/hypium';
import { util } from '@kit.ArkTS';
import { naturalSort } from 'commons';
import {
  DisplayRect,
  FontSizeMode,
  LayoutEngine,
  RenderOptions,
  RenderOrientation,
  VerticalDirection,
  containFit,
  pixelToDisplay
} from 'corerender';
import { BingTranslator, SseParser } from 'coretranslate';
import { makePixelRect } from 'coremodel';

function toBuffer(bytes: Array<number>): ArrayBuffer {
  const arr = new Uint8Array(bytes.length);
  for (let i = 0; i < bytes.length; i++) {
    arr[i] = bytes[i] & 0xFF;
  }
  return arr.buffer;
}

function utf8Bytes(s: string): Array<number> {
  const encoded = new util.TextEncoder().encodeInto(s);
  const out: Array<number> = [];
  for (let i = 0; i < encoded.length; i++) {
    out.push(encoded[i]);
  }
  return out;
}

function renderOptions(orientation: RenderOrientation, mode: FontSizeMode, size: number):
RenderOptions {
  const o: RenderOptions = {
    orientation: orientation,
    verticalDirection: VerticalDirection.RTL,
```

```
    fontSizeMode: mode,
    fixedFontSize: size,
    padding: 4
  };
  return o;
}

function makeBox(w: number, h: number): DisplayRect {
  const box: DisplayRect = { x: 0, y: 0, w: w, h: h };
  return box;
}

export default function localUnitTest() {
  describe('Moyi local unit tests', () => {
    // ---------- T044: coordinate mapping ----------
    it('containFit_scales_and_centers', 0, () => {
      const fit = containFit(100, 200, 100, 100);
      expect(fit.scale).assertEqual(0.5);
      expect(fit.dx).assertEqual(25);
      expect(fit.dy).assertEqual(0);
    });

    it('pixelToDisplay_maps_box', 0, () => {
      const rect = pixelToDisplay(makePixelRect(10, 20, 30, 40), 0.5, 5, 5);
      expect(rect.x).assertEqual(10);
      expect(rect.y).assertEqual(15);
      expect(rect.w).assertEqual(15);
      expect(rect.h).assertEqual(20);
    });

    // ---------- T044: natural sort (v2 < v10) ----------
    it('naturalSort_numeric_aware', 0, () => {
      const sorted = naturalSort(['v10.jpg', 'v2.jpg', 'v1.jpg', 'v20.jpg']);
      expect(sorted[0]).assertEqual('v1.jpg');
      expect(sorted[1]).assertEqual('v2.jpg');
      expect(sorted[2]).assertEqual('v10.jpg');
      expect(sorted[3]).assertEqual('v20.jpg');
    });

    it('naturalSort_mixed_names', 0, () => {
      const sorted = naturalSort(['page-2.png', 'page-10.png', 'cover.png']);
      expect(sorted[0]).assertEqual('cover.png');
      expect(sorted[1]).assertEqual('page-2.png');
      expect(sorted[2]).assertEqual('page-10.png');
    });

    // ---------- T044: layout / ellipsis / font auto ----------
    it('layout_horizontal_auto_font_nonzero', 0, () => {
      const layout = LayoutEngine.layout('好', makeBox(120, 120),
        renderOptions(RenderOrientation.HORIZONTAL, FontSizeMode.AUTO, 16));
```

```
    expect(layout.runs.length).assertEqual(1);
    expect(layout.measuredFontSize >= 10).assertTrue();
  });

  it('layout_vertical_runs_per_char', 0, () => {
    const layout = LayoutEngine.layout('你好世界', makeBox(80, 240),
      renderOptions(RenderOrientation.VERTICAL, FontSizeMode.AUTO, 16));
    expect(layout.runs.length).assertEqual(4);
  });

  it('layout_fixed_font_overflow_truncates_with_ellipsis', 0, () => {
    const opts = renderOptions(RenderOrientation.HORIZONTAL, FontSizeMode.FIXED, 24);
    const layout = LayoutEngine.layout('这是一段很长很长的译文需要被截断处理不能拆行', makeBox(40, 28), opts);
    expect(layout.truncated).assertTrue();
    let last = '';
    if (layout.runs.length > 0) {
      last = layout.runs[layout.runs.length - 1].char;
    }
    expect(last).assertEqual('…');
  });

  it('layout_vertical_rtl_column_order', 0, () => {
    const opts = renderOptions(RenderOrientation.VERTICAL, FontSizeMode.FIXED, 20);
    const layout = LayoutEngine.layout('你好世界一二', makeBox(40, 60), opts);
    expect(layout.runs.length > 0).assertTrue();
  });

  // ---------- T045: SSE parsing ----------
  it('sse_single_event_and_done', 0, () => {
    const events: Array<string> = [];
    let done = false;
    const parser = new SseParser({
      onEvent: (data: string) => {
        events.push(data);
      },
      onDone: () => {
        done = true;
      }
    });
    parser.push(toBuffer(utf8Bytes('data: hello\n\n')));
    parser.push(toBuffer(utf8Bytes('data: [DONE]\n\n')));
    expect(events.length).assertEqual(1);
    expect(events[0]).assertEqual('hello');
    expect(done).assertTrue();
  });

  it('sse_multi_event_framing', 0, () => {
    const events: Array<string> = [];
    const parser = new SseParser({
      onEvent: (data: string) => {
```

```
          events.push(data);
        },
        onDone: () => {
        }
      });
      parser.push(toBuffer(utf8Bytes('data: a\n\ndata: b\n\n')));
      expect(events.length).assertEqual(2);
      expect(events[0]).assertEqual('a');
      expect(events[1]).assertEqual('b');
    });

    it('sse_cross_chunk_utf8_safe', 0, () => {
      const events: Array<string> = [];
      const parser = new SseParser({
        onEvent: (data: string) => {
          events.push(data);
        },
        onDone: () => {
        }
      });
      // "data: 日\n\n" — 日 = E6 97 A5, split so the 3 bytes span two chunks.
      const bytes = utf8Bytes('data: 日\n\n');
      const split = 7;
      parser.push(toBuffer(bytes.slice(0, split)));
      parser.push(toBuffer(bytes.slice(split)));
      expect(events.length).assertEqual(1);
      expect(events[0]).assertEqual('日');
    });

    // ---------- T045: Bing request assembly ----------
    it('bing_form_body_shape', 0, () => {
      const body = BingTranslator.buildFormBody('zh', 'en', '你好', 'tok123', 'key456');
      expect(body.indexOf('fromLang=zh-Hans') >= 0).assertTrue();
      expect(body.indexOf('to=en') >= 0).assertTrue();
      expect(body.indexOf(`text=${encodeURIComponent('你好')}`) >= 0).assertTrue();
      expect(body.indexOf('token=tok123') >= 0).assertTrue();
      expect(body.indexOf('key=key456') >= 0).assertTrue();
    });
  });
}
// ==================== 文件: entry/src/main/cpp/llamacpp_bridge.cpp ====================
// MoyiOhos llama.cpp NAPI bridge (T041 / plan D11).
// Exposes: getVersion / loadModel / generate(streaming via
// napi_threadsafe_function) / stopGeneration / unloadModel.
//
// Generation runs on a worker std::thread; each decoded token is
// marshalled back to the ArkTS thread through a threadsafe function
// (逐 token 回调). A handle is returned by loadModel and used by the
// other calls.
```

```
#include <napi/native_api.h>

#include <atomic>
#include <cstring>
#include <mutex>
#include <string>
#include <thread>
#include <unordered_map>
#include <vector>

#include "llama.h"

namespace {

struct ModelEntry {
    llama_model * model = nullptr;
    llama_context * ctx = nullptr;
    std::atomic<bool> stop{false};
    std::atomic<bool> busy{false};
};

std::mutex g_mutex;
std::unordered_map<int, ModelEntry *> g_entries;
int g_next_handle = 1;

struct GenCallbacks {
    napi_ref on_token = nullptr;
    napi_ref on_done = nullptr;
};

struct GenMsg {
    bool done = false;
    bool is_error = false;
    std::string text;
};

void finalize_callbacks(napi_env env, void * data, void * hint) {
    GenCallbacks * cbs = static_cast<GenCallbacks *>(data);
    if (cbs == nullptr) {
        return;
    }
    if (cbs->on_token != nullptr) {
        napi_delete_reference(env, cbs->on_token);
    }
    if (cbs->on_done != nullptr) {
        napi_delete_reference(env, cbs->on_done);
    }
    delete cbs;
}
```

```
void call_js(napi_env env, napi_value js_cb, void * context, void * data) {
    GenCallbacks * cbs = static_cast<GenCallbacks *>(context);
    GenMsg * msg = static_cast<GenMsg *>(data);
    if (cbs == nullptr || msg == nullptr) {
        delete msg;
        return;
    }
    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    napi_value fn = nullptr;
    if (msg->done) {
        if (cbs->on_done != nullptr) {
            napi_get_reference_value(env, cbs->on_done, &fn);
        }
    } else {
        if (cbs->on_token != nullptr) {
            napi_get_reference_value(env, cbs->on_token, &fn);
        }
    }
    if (fn != nullptr) {
        napi_value arg = nullptr;
        napi_create_string_utf8(env, msg->text.c_str(), msg->text.size(), &arg);
        if (msg->done) {
            napi_value errArg = nullptr;
            napi_get_boolean(env, msg->is_error, &errArg);
            napi_value args[2] = {arg, errArg};
            napi_call_function(env, undefined, fn, 2, args, nullptr);
        } else {
            napi_call_function(env, undefined, fn, 1, &arg, nullptr);
        }
    }
    delete msg;
}

void run_generation(napi_env env, napi_threadsafe_function tsfn, ModelEntry * entry,
                    std::string prompt, int maxTokens, float temperature) {
    bool error = false;
    std::string error_text;
    const llama_vocab * vocab = llama_model_get_vocab(entry->model);

    std::vector<llama_token> tokens(prompt.size() + 8);
    int n = llama_tokenize(vocab, prompt.c_str(), static_cast<int32_t>(prompt.size()),
                           tokens.data(), static_cast<int32_t>(tokens.size()), true, true);
    if (n < 0) {
        tokens.resize(-n);
        n = llama_tokenize(vocab, prompt.c_str(), static_cast<int32_t>(prompt.size()),
                           tokens.data(), static_cast<int32_t>(tokens.size()), true, true);
    }
    if (n <= 0) {
        GenMsg * msg = new GenMsg();
```

```
        msg->done = true;
        msg->is_error = true;
        msg->text = "提示词分词失败";
        napi_call_threadsafe_function(tsfn, msg, napi_tsfn_blocking);
        napi_release_threadsafe_function(tsfn, napi_tsfn_release);
        return;
    }

    llama_sampler_chain_params sparams = llama_sampler_chain_default_params();
    llama_sampler * smpl = llama_sampler_chain_init(sparams);
    llama_sampler_chain_add(smpl, llama_sampler_init_top_k(40));
    llama_sampler_chain_add(smpl, llama_sampler_init_top_p(0.9, 1));
    llama_sampler_chain_add(smpl, llama_sampler_init_temp(temperature));
    llama_sampler_chain_add(smpl, llama_sampler_init_dist(LLAMA_DEFAULT_SEED));

    const int n_batch = 512;
    for (int i = 0; i < n; i += n_batch) {
        int count = (n - i) < n_batch ? (n - i) : n_batch;
        llama_batch batch = llama_batch_get_one(tokens.data() + i, count);
        if (llama_decode(entry->ctx, batch) != 0) {
            error = true;
            error_text = "提示词解码失败（模型或上下文不兼容）";
            break;
        }
    }

    std::string full;
    char buf[256];
    if (!error) {
        for (int produced = 0; produced < maxTokens; produced++) {
            if (entry->stop.load()) {
                break;
            }
            llama_token id = llama_sampler_sample(smpl, entry->ctx, -1);
            if (llama_vocab_is_eog(vocab, id)) {
                break;
            }
            int piece_len = llama_token_to_piece(vocab, id, buf, sizeof(buf), 0, false);
            if (piece_len > 0) {
                std::string piece(buf, piece_len);
                full += piece;
                GenMsg * msg = new GenMsg();
                msg->text = piece;
                napi_call_threadsafe_function(tsfn, msg, napi_tsfn_blocking);
            }
            llama_sampler_accept(smpl, id);
            llama_batch batch = llama_batch_get_one(&id, 1);
            if (llama_decode(entry->ctx, batch) != 0) {
                error = true;
                error_text = "推理中断（decode 失败）";
```

```
                break;
            }
        }
    }

    llama_sampler_free(smpl);
    entry->busy.store(false);

    GenMsg * done = new GenMsg();
    done->done = true;
    done->is_error = error;
    done->text = error ? error_text : full;
    napi_call_threadsafe_function(tsfn, done, napi_tsfn_blocking);
    napi_release_threadsafe_function(tsfn, napi_tsfn_release);
}

napi_value get_version(napi_env env, napi_callback_info info) {
    napi_value result = nullptr;
    const char * text = "llama.cpp (MoyiOhos NAPI bridge)";
    napi_create_string_utf8(env, text, NAPI_AUTO_LENGTH, &result);
    return result;
}

napi_value load_model(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 1) {
        napi_throw_error(env, nullptr, "loadModel 需要模型路径");
        return nullptr;
    }
    char path[1024];
    size_t len = 0;
    if (napi_get_value_string_utf8(env, args[0], path, sizeof(path), &len) != napi_ok) {
        napi_throw_error(env, nullptr, "模型路径无效");
        return nullptr;
    }

    llama_backend_init();
    llama_model_params mparams = llama_model_default_params();
    llama_model * model = llama_model_load_from_file(path, mparams);
    if (model == nullptr) {
        napi_throw_error(env, nullptr, "模型加载失败：文件损坏或量化类型不受支持（含 1.25bit STQ1_0 本版暂不支持）");
        return nullptr;
    }
    llama_context_params cparams = llama_context_default_params();
    cparams.n_ctx = 2048;
    llama_context * ctx = llama_init_from_model(model, cparams);
    if (ctx == nullptr) {
        llama_model_free(model);
```

```
        napi_throw_error(env, nullptr, "上下文创建失败（内存不足或模型不兼容）");
        return nullptr;
    }

    ModelEntry * entry = new ModelEntry();
    entry->model = model;
    entry->ctx = ctx;

    std::lock_guard<std::mutex> lock(g_mutex);
    int handle = g_next_handle++;
    g_entries[handle] = entry;

    napi_value result = nullptr;
    napi_create_int32(env, handle, &result);
    return result;
}

napi_value generate(napi_env env, napi_callback_info info) {
    size_t argc = 6;
    napi_value args[6] = {nullptr, nullptr, nullptr, nullptr, nullptr, nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 6) {
        napi_throw_error(env, nullptr, "generate 参数不足");
        return nullptr;
    }
    int32_t handle = 0;
    napi_get_value_int32(env, args[0], &handle);

    std::string prompt;
    size_t plen = 0;
    napi_get_value_string_utf8(env, args[1], nullptr, 0, &plen);
    prompt.resize(plen);
    napi_get_value_string_utf8(env, args[1], &prompt[0], plen + 1, &plen);

    int32_t maxTokens = 128;
    napi_get_value_int32(env, args[2], &maxTokens);
    double temperature = 0.3;
    napi_get_value_double(env, args[3], &temperature);

    ModelEntry * entry = nullptr;
    {
        std::lock_guard<std::mutex> lock(g_mutex);
        auto it = g_entries.find(handle);
        if (it != g_entries.end()) {
            entry = it->second;
        }
    }
    if (entry == nullptr) {
        napi_throw_error(env, nullptr, "模型句柄无效或已卸载");
        return nullptr;
```

```
    }
    if (entry->busy.load()) {
        napi_throw_error(env, nullptr, "该模型正在生成中");
        return nullptr;
    }
    entry->busy.store(true);
    entry->stop.store(false);

    GenCallbacks * cbs = new GenCallbacks();
    napi_create_reference(env, args[4], 1, &cbs->on_token);
    napi_create_reference(env, args[5], 1, &cbs->on_done);

    napi_value resource_name = nullptr;
    napi_create_string_utf8(env, "llamacpp_generate", NAPI_AUTO_LENGTH, &resource_name);

    napi_threadsafe_function tsfn = nullptr;
    napi_status status = napi_create_threadsafe_function(
        env, nullptr, nullptr, resource_name, 0, 1, nullptr, finalize_callbacks, cbs, call_js,
&tsfn);
    if (status != napi_ok) {
        entry->busy.store(false);
        napi_delete_reference(env, cbs->on_token);
        napi_delete_reference(env, cbs->on_done);
        delete cbs;
        napi_throw_error(env, nullptr, "线程安全回调创建失败");
        return nullptr;
    }

    std::thread worker(run_generation, env, tsfn, entry, prompt, static_cast<int>(maxTokens),
                       static_cast<float>(temperature));
    worker.detach();

    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    return undefined;
}

napi_value stop_generation(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    int32_t handle = 0;
    if (argc >= 1) {
        napi_get_value_int32(env, args[0], &handle);
    }
    std::lock_guard<std::mutex> lock(g_mutex);
    auto it = g_entries.find(handle);
    if (it != g_entries.end()) {
        it->second->stop.store(true);
    }
```

```
    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    return undefined;
}

napi_value unload_model(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    int32_t handle = 0;
    if (argc >= 1) {
        napi_get_value_int32(env, args[0], &handle);
    }
    ModelEntry * entry = nullptr;
    {
        std::lock_guard<std::mutex> lock(g_mutex);
        auto it = g_entries.find(handle);
        if (it != g_entries.end()) {
            entry = it->second;
            g_entries.erase(it);
        }
    }
    if (entry != nullptr) {
        entry->stop.store(true);
        if (entry->ctx != nullptr) {
            llama_free(entry->ctx);
        }
        if (entry->model != nullptr) {
            llama_model_free(entry->model);
        }
        delete entry;
    }
    napi_value undefined = nullptr;
    napi_get_undefined(env, &undefined);
    return undefined;
}

} // namespace

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        {"getVersion", nullptr, get_version, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"loadModel", nullptr, load_model, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"generate", nullptr, generate, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"stopGeneration", nullptr, stop_generation, nullptr, nullptr, nullptr, napi_default,
nullptr},
        {"unloadModel", nullptr, unload_model, nullptr, nullptr, nullptr, napi_default,
nullptr},
    };
```

```
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]), desc);
    return exports;
}
EXTERN_C_END

static napi_module g_llamacpp_module = {
    .nm_version = 1,
    .nm_flags = 0,
    .nm_filename = nullptr,
    .nm_register_func = Init,
    .nm_modname = "llamacpp",
    .nm_priv = nullptr,
    .reserved = {0},
};

extern "C" __attribute__((constructor)) void RegisterLlamaCppModule(void) {
    napi_module_register(&g_llamacpp_module);
}
// ==================== 文件: entry/src/main/cpp/ocr_bridge.cpp ====================
// Native async-capable PP-OCR rec bridge: ONNX Runtime C API.
// Det stays in ArkTS; this module only runs rec boxes on the calling thread,
// so ArkTS can invoke it from TaskPool/Worker without JS ORT session issues.

#include <napi/native_api.h>
#include "onnxruntime_c_api.h"

#include <algorithm>
#include <cmath>
#include <cstring>
#include <mutex>
#include <string>
#include <vector>

namespace {

const OrtApi * g_api = nullptr;
OrtEnv * g_env = nullptr;
OrtSession * g_session = nullptr;
OrtSession * g_det_session = nullptr;
OrtSessionOptions * g_options = nullptr;
OrtMemoryInfo * g_memory = nullptr;
OrtAllocator * g_allocator = nullptr;
std::mutex g_mutex;
std::string g_input_name;
std::string g_output_name;
std::string g_det_input_name;
std::string g_det_output_name;

bool ensure_api() {
    if (g_api == nullptr) {
```

```
        g_api = OrtGetApiBase()->GetApi(ORT_API_VERSION);
    }
    return g_api != nullptr;
}

bool ensure_env() {
    if (!ensure_api()) return false;
    if (g_env == nullptr) {
        if (g_api->CreateEnv(ORT_LOGGING_LEVEL_WARNING, "MoyiOcr", &g_env) != nullptr) {
            g_env = nullptr;
            return false;
        }
        if (g_api->GetAllocatorWithDefaultOptions(&g_allocator) != nullptr) return false;
        if (g_api->CreateCpuMemoryInfo(OrtArenaAllocator, OrtMemTypeDefault, &g_memory) !=
nullptr) return false;
    }
    return true;
}

bool init_rec(const std::string & model_path) {
    std::lock_guard<std::mutex> lock(g_mutex);
    if (!ensure_env()) return false;
    if (g_session != nullptr) {
        g_api->ReleaseSession(g_session);
        g_session = nullptr;
    }
    if (g_options == nullptr) {
        if (g_api->CreateSessionOptions(&g_options) != nullptr) return false;
        g_api->SetIntraOpNumThreads(g_options, 2);
        g_api->SetSessionGraphOptimizationLevel(g_options, ORT_ENABLE_ALL);
    }
    if (g_api->CreateSession(g_env, model_path.c_str(), g_options, &g_session) != nullptr) {
        g_session = nullptr;
        return false;
    }
    char * name = nullptr;
    if (g_api->SessionGetInputName(g_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_input_name = name;
    g_allocator->Free(g_allocator, name);
    name = nullptr;
    if (g_api->SessionGetOutputName(g_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_output_name = name;
    g_allocator->Free(g_allocator, name);
    return true;
}

inline void sample_rgb(const unsigned char * rgba, int w, int h, float fx, float fy, float
out[3]);
```

```
bool init_det(const std::string & model_path) {
    std::lock_guard<std::mutex> lock(g_mutex);
    if (!ensure_env()) return false;
    if (g_det_session != nullptr) {
        g_api->ReleaseSession(g_det_session);
        g_det_session = nullptr;
    }
    if (g_options == nullptr) {
        if (g_api->CreateSessionOptions(&g_options) != nullptr) return false;
        g_api->SetIntraOpNumThreads(g_options, 2);
        g_api->SetSessionGraphOptimizationLevel(g_options, ORT_ENABLE_ALL);
    }
    if (g_api->CreateSession(g_env, model_path.c_str(), g_options, &g_det_session) != nullptr) {
        g_det_session = nullptr;
        return false;
    }
    char * name = nullptr;
    if (g_api->SessionGetInputName(g_det_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_det_input_name = name;
    g_allocator->Free(g_allocator, name);
    name = nullptr;
    if (g_api->SessionGetOutputName(g_det_session, 0, g_allocator, &name) != nullptr || name ==
nullptr) return false;
    g_det_output_name = name;
    g_allocator->Free(g_allocator, name);
    return true;
}

struct DetBox { int x; int y; int w; int h; };

std::vector<DetBox> detect_boxes(const unsigned char * rgba, int w, int h) {
    std::vector<DetBox> result;
    if (g_det_session == nullptr || g_api == nullptr || w <= 0 || h <= 0) return result;
    const int DET_MAX_SIDE = 768;
    float scale = 1.0f;
    int max_side = std::max(w, h);
    if (max_side > DET_MAX_SIDE) scale = (float)DET_MAX_SIDE / (float)max_side;
    int detW = std::max(32, (int)std::round((float)w * scale / 32.0f) * 32);
    int detH = std::max(32, (int)std::round((float)h * scale / 32.0f) * 32);
    std::vector<float> input((size_t)3 * detH * detW, 0.0f);
    size_t plane = (size_t)detW * detH;
    for (int y = 0; y < detH; ++y) {
        float sy = (y + 0.5f) * h / detH - 0.5f;
        for (int x = 0; x < detW; ++x) {
            float sx = (x + 0.5f) * w / detW - 0.5f;
            float rgb[3];
            sample_rgb(rgba, w, h, sx, sy, rgb);
            size_t idx = (size_t)y * detW + x;
```

```
            input[idx] = (rgb[0] / 255.0f - 0.5f) / 0.5f;
            input[plane + idx] = (rgb[1] / 255.0f - 0.5f) / 0.5f;
            input[plane * 2 + idx] = (rgb[2] / 255.0f - 0.5f) / 0.5f;
        }
    }
    int64_t dims[4] = {1, 3, detH, detW};
    OrtValue * tensor = nullptr;
    if (g_api->CreateTensorWithDataAsOrtValue(g_memory, input.data(), input.size() *
sizeof(float),
        dims, 4, ONNX_TENSOR_ELEMENT_DATA_TYPE_FLOAT, &tensor) != nullptr) {
        return result;
    }
    const char * input_names[1] = {g_det_input_name.c_str()};
    const char * output_names[1] = {g_det_output_name.c_str()};
    OrtValue * output = nullptr;
    if (g_api->Run(g_det_session, nullptr, input_names, &tensor, 1, output_names, 1, &output) ==
 nullptr && output != nullptr) {
        OrtTensorTypeAndShapeInfo * info = nullptr;
        if (g_api->GetTensorTypeAndShape(output, &info) == nullptr && info != nullptr) {
            size_t count = 0;
            g_api->GetDimensionsCount(info, &count);
            std::vector<int64_t> shape(count);
            if (count >= 2) g_api->GetDimensions(info, shape.data(), count);
            float * data = nullptr;
            if (count >= 2 && g_api->GetTensorMutableData(output, (void **)&data) == nullptr &&
data != nullptr) {
                int predH = (int)shape[count - 2];
                int predW = (int)shape[count - 1];
                std::vector<unsigned char> mask((size_t)predW * predH, 0);
                for (size_t i = 0; i < mask.size(); ++i) mask[i] = data[i] > 0.3f ? 1 : 0;
                std::vector<unsigned char> tmp(mask.size(), 0);
                const int radius = 3;
                for (int y = 0; y < predH; ++y) {
                    for (int x = 0; x < predW; ++x) {
                        if (!mask[(size_t)y * predW + x]) continue;
                        int x0 = std::max(0, x - radius), x1 = std::min(predW - 1, x + radius);
                        for (int nx = x0; nx <= x1; ++nx) tmp[(size_t)y * predW + nx] = 1;
                    }
                }
                std::vector<unsigned char> dilated(mask.size(), 0);
                for (int y = 0; y < predH; ++y) {
                    for (int x = 0; x < predW; ++x) {
                        if (!tmp[(size_t)y * predW + x]) continue;
                        int y0 = std::max(0, y - radius), y1 = std::min(predH - 1, y + radius);
                        for (int ny = y0; ny <= y1; ++ny) dilated[(size_t)ny * predW + x] = 1;
                    }
                }
                std::vector<unsigned char> visited(dilated.size(), 0);
                std::vector<int> stack;
                float sx = (float)w / predW;
```

```
                float sy = (float)h / predH;
                for (int i = 0; i < (int)dilated.size(); ++i) {
                    if (!dilated[i] || visited[i]) continue;
                    int minX = predW, maxX = 0, minY = predH, maxY = 0, cc = 0;
                    stack.push_back(i);
                    visited[i] = 1;
                    while (!stack.empty()) {
                        int idx = stack.back();
                        stack.pop_back();
                        int x = idx % predW;
                        int y = idx / predW;
                        ++cc;
                        minX = std::min(minX, x); maxX = std::max(maxX, x);
                        minY = std::min(minY, y); maxY = std::max(maxY, y);
                        if (x > 0 && dilated[idx - 1] && !visited[idx - 1]) { visited[idx - 1] =
 1; stack.push_back(idx - 1); }
                        if (x + 1 < predW && dilated[idx + 1] && !visited[idx + 1]) {
visited[idx + 1] = 1; stack.push_back(idx + 1); }
                        if (y > 0 && dilated[idx - predW] && !visited[idx - predW]) {
visited[idx - predW] = 1; stack.push_back(idx - predW); }
                        if (y + 1 < predH && dilated[idx + predW] && !visited[idx + predW]) {
visited[idx + predW] = 1; stack.push_back(idx + predW); }
                    }
                    if (cc < 12) continue;
                    DetBox box;
                    box.x = std::max(0, (int)std::floor(minX * sx));
                    box.y = std::max(0, (int)std::floor(minY * sy));
                    box.w = std::min(w - box.x, (int)std::ceil((maxX - minX + 1) * sx));
                    box.h = std::min(h - box.y, (int)std::ceil((maxY - minY + 1) * sy));
                    if (box.w < 8 || box.h < 8) continue;
                    int ex = (int)std::round(box.w * 0.12f);
                    int ey = (int)std::round(box.h * 0.12f);
                    box.x = std::max(0, box.x - ex);
                    box.y = std::max(0, box.y - ey);
                    box.w = std::min(w - box.x, box.w + ex * 2);
                    box.h = std::min(h - box.y, box.h + ey * 2);
                    result.push_back(box);
                    if (result.size() >= 80) break;
                }
                bool changed = true;
                while (changed) {
                    changed = false;
                    for (size_t i = 0; i < result.size() && !changed; ++i) {
                        for (size_t j = i + 1; j < result.size(); ++j) {
                            const DetBox & a = result[i];
                            const DetBox & b = result[j];
                            const int gap = 8;
                            bool near = a.x - gap <= b.x + b.w && b.x - gap <= a.x + a.w
                                && a.y - gap <= b.y + b.h && b.y - gap <= a.y + a.h;
                            if (near) {
```

```
                                int x0 = std::min(a.x, b.x), y0 = std::min(a.y, b.y);
                                int x1 = std::max(a.x + a.w, b.x + b.w), y1 = std::max(a.y +
a.h, b.y + b.h);
                                result[i].x = x0; result[i].y = y0; result[i].w = x1 - x0;
result[i].h = y1 - y0;
                                result.erase(result.begin() + j);
                                changed = true;
                                break;
                            }
                        }
                    }
                }
            }
            g_api->ReleaseTensorTypeAndShapeInfo(info);
        }
    }
    if (output != nullptr) g_api->ReleaseValue(output);
    g_api->ReleaseValue(tensor);
    return result;
}

inline void sample_rgb(const unsigned char * rgba, int w, int h, float fx, float fy, float
out[3]) {
    float x = std::max(0.0f, std::min((float)w - 1.0f, fx));
    float y = std::max(0.0f, std::min((float)h - 1.0f, fy));
    int x0 = (int)std::floor(x);
    int y0 = (int)std::floor(y);
    int x1 = std::min(w - 1, x0 + 1);
    int y1 = std::min(h - 1, y0 + 1);
    float wx = x - x0;
    float wy = y - y0;
    for (int c = 0; c < 3; ++c) {
        float top = rgba[(y0 * w + x0) * 4 + c] * (1.0f - wx) + rgba[(y0 * w + x1) * 4 + c] *
wx;
        float bottom = rgba[(y1 * w + x0) * 4 + c] * (1.0f - wx) + rgba[(y1 * w + x1) * 4 + c] *
 wx;
        out[c] = top * (1.0f - wy) + bottom * wy;
    }
}

std::string rec_one(const unsigned char * rgba, int w, int h,
                    int bx, int by, int bw, int bh, const std::vector<std::string> & dict) {
    const int REC_H = 48;
    int pad = 2;
    int x0 = std::max(0, bx - pad);
    int y0 = std::max(0, by - pad);
    int x1 = std::min(w, bx + bw + pad);
    int y1 = std::min(h, by + bh + pad);
    int cw = std::max(1, x1 - x0);
    int ch = std::max(1, y1 - y0);
```

```
    int target_w = (int)std::ceil((float)REC_H * cw / ch);
    target_w = std::max(32, (int)std::ceil(target_w / 32.0f) * 32);
    target_w = std::min(target_w, 1280);
    std::vector<float> input((size_t)3 * REC_H * target_w, 0.0f);
    int used_w = (int)std::ceil((float)REC_H * cw / ch);
    for (int y = 0; y < REC_H; ++y) {
        float sy = y0 + (y + 0.5f) * ch / REC_H - 0.5f;
        for (int x = 0; x < target_w; ++x) {
            if (x >= used_w) continue;
            float sx = x0 + (x + 0.5f) * cw / used_w - 0.5f;
            float rgb[3];
            sample_rgb(rgba, w, h, sx, sy, rgb);
            size_t base = (size_t)y * target_w + x;
            input[base] = (rgb[0] / 255.0f - 0.5f) / 0.5f;
            input[REC_H * target_w + base] = (rgb[1] / 255.0f - 0.5f) / 0.5f;
            input[2 * REC_H * target_w + base] = (rgb[2] / 255.0f - 0.5f) / 0.5f;
        }
    }
    int64_t dims[4] = {1, 3, REC_H, target_w};
    OrtValue * tensor = nullptr;
    if (g_api->CreateTensorWithDataAsOrtValue(g_memory, input.data(), input.size() *
sizeof(float),
        dims, 4, ONNX_TENSOR_ELEMENT_DATA_TYPE_FLOAT, &tensor) != nullptr) {
        return "";
    }
    const char * input_names[1] = {g_input_name.c_str()};
    const char * output_names[1] = {g_output_name.c_str()};
    OrtValue * output = nullptr;
    std::string text;
    if (g_api->Run(g_session, nullptr, input_names, &tensor, 1, output_names, 1, &output) ==
nullptr && output != nullptr) {
        OrtTensorTypeAndShapeInfo * info = nullptr;
        if (g_api->GetTensorTypeAndShape(output, &info) == nullptr && info != nullptr) {
            size_t count = 0;
            g_api->GetDimensionsCount(info, &count);
            std::vector<int64_t> shape(count);
            if (count > 0) g_api->GetDimensions(info, shape.data(), count);
            float * data = nullptr;
            if (g_api->GetTensorMutableData(output, (void **)&data) == nullptr && data !=
nullptr && count >= 2) {
                int seq = (int)shape[count - 2];
                int classes = (int)shape[count - 1];
                int prev = -1;
                for (int t = 0; t < seq; ++t) {
                    int best = 0;
                    float prob = data[(size_t)t * classes];
                    for (int c = 1; c < classes; ++c) {
                        float p = data[(size_t)t * classes + c];
                        if (p > prob) { prob = p; best = c; }
                    }
```

```
                    if (best > 0 && best != prev && best < (int)dict.size()) text += dict[best];
                    prev = best;
                }
            }
            g_api->ReleaseTensorTypeAndShapeInfo(info);
        }
    }
    if (output != nullptr) g_api->ReleaseValue(output);
    g_api->ReleaseValue(tensor);
    return text;
}

napi_value init_rec_js(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 1) { napi_throw_error(env, nullptr, "initRec 需要模型路径"); return nullptr; }
    char path[1024];
    size_t len = 0;
    if (napi_get_value_string_utf8(env, args[0], path, sizeof(path), &len) != napi_ok) return
nullptr;
    bool ok = init_rec(std::string(path, len));
    napi_value result = nullptr;
    napi_get_boolean(env, ok, &result);
    return result;
}

napi_value recognize_boxes(napi_env env, napi_callback_info info) {
    size_t argc = 5;
    napi_value args[5] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 5) { napi_throw_error(env, nullptr, "recognizeBoxes 参数不足"); return nullptr; }
    void * raw = nullptr;
    size_t raw_len = 0;
    if (napi_get_arraybuffer_info(env, args[0], &raw, &raw_len) != napi_ok) return nullptr;
    int32_t w = 0, h = 0;
    napi_get_value_int32(env, args[1], &w);
    napi_get_value_int32(env, args[2], &h);
    uint32_t box_count = 0;
    napi_get_array_length(env, args[3], &box_count);
    std::vector<int> boxes(box_count);
    for (uint32_t i = 0; i < box_count; ++i) {
        napi_value item = nullptr;
        napi_get_element(env, args[3], i, &item);
        int32_t v = 0;
        napi_get_value_int32(env, item, &v);
        boxes[i] = v;
    }
    size_t dict_len = 0;
    napi_get_value_string_utf8(env, args[4], nullptr, 0, &dict_len);
```

```
    std::string dict_text(dict_len, '\0');
    if (dict_len > 0) napi_get_value_string_utf8(env, args[4], &dict_text[0], dict_len + 1,
&dict_len);
    std::vector<std::string> dict;
    dict.push_back("");
    size_t start = 0;
    for (size_t i = 0; i <= dict_text.size(); ++i) {
        if (i == dict_text.size() || dict_text[i] == '\n') {
            std::string line = dict_text.substr(start, i - start);
            if (!line.empty() && line.back() == '\r') line.pop_back();
            if (!line.empty()) dict.push_back(line);
            start = i + 1;
        }
    }
    std::lock_guard<std::mutex> lock(g_mutex);
    napi_value arr = nullptr;
    napi_create_array_with_length(env, box_count / 4, &arr);
    if (g_session != nullptr && w > 0 && h > 0) {
        const unsigned char * rgba = (const unsigned char *)raw;
        for (uint32_t i = 0, j = 0; i + 3 < box_count; i += 4, ++j) {
            std::string text = rec_one(rgba, w, h, boxes[i], boxes[i + 1], boxes[i + 2], boxes[i
 + 3], dict);
            napi_value s = nullptr;
            napi_create_string_utf8(env, text.c_str(), text.size(), &s);
            napi_set_element(env, arr, j, s);
        }
    }
    return arr;
}

napi_value init_det_js(napi_env env, napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 1) { napi_throw_error(env, nullptr, "initDet 需要模型路径"); return nullptr; }
    char path[1024];
    size_t len = 0;
    if (napi_get_value_string_utf8(env, args[0], path, sizeof(path), &len) != napi_ok) return
nullptr;
    bool ok = init_det(std::string(path, len));
    napi_value result = nullptr;
    napi_get_boolean(env, ok, &result);
    return result;
}

napi_value detect_js(napi_env env, napi_callback_info info) {
    size_t argc = 3;
    napi_value args[3] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);
    if (argc < 3) { napi_throw_error(env, nullptr, "detectBoxes 参数不足"); return nullptr; }
```

```
    void * raw = nullptr;
    size_t raw_len = 0;
    if (napi_get_arraybuffer_info(env, args[0], &raw, &raw_len) != napi_ok) return nullptr;
    int32_t w = 0, h = 0;
    napi_get_value_int32(env, args[1], &w);
    napi_get_value_int32(env, args[2], &h);
    std::lock_guard<std::mutex> lock(g_mutex);
    std::vector<DetBox> boxes = detect_boxes((const unsigned char *)raw, w, h);
    napi_value arr = nullptr;
    napi_create_array_with_length(env, boxes.size() * 4, &arr);
    uint32_t k = 0;
    for (const auto & b : boxes) {
        napi_value v = nullptr;
        napi_create_int32(env, b.x, &v); napi_set_element(env, arr, k++, v);
        napi_create_int32(env, b.y, &v); napi_set_element(env, arr, k++, v);
        napi_create_int32(env, b.w, &v); napi_set_element(env, arr, k++, v);
        napi_create_int32(env, b.h, &v); napi_set_element(env, arr, k++, v);
    }
    return arr;
}

} // namespace

EXTERN_C_START
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        {"initRec", nullptr, init_rec_js, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"initDet", nullptr, init_det_js, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"detectBoxes", nullptr, detect_js, nullptr, nullptr, nullptr, napi_default, nullptr},
        {"recognizeBoxes", nullptr, recognize_boxes, nullptr, nullptr, nullptr, napi_default,
nullptr},
    };
    napi_define_properties(env, exports, sizeof(desc) / sizeof(desc[0]), desc);
    return exports;
}
EXTERN_C_END

static napi_module g_ocr_module = {
    .nm_version = 1,
    .nm_flags = 0,
    .nm_filename = nullptr,
    .nm_register_func = Init,
    .nm_modname = "ocrasync",
    .nm_priv = nullptr,
    .reserved = {0},
};

extern "C" __attribute__((constructor)) void RegisterOcrModule(void) {
    napi_module_register(&g_ocr_module);
}
```